TOOP: A TRANSFER OF OWNERSHIP PROTOCOL OVER BITCOIN

A. FUTORANSKY, F. BARBÀRA, R. FERNÁNDEZ, G. LAROTONDA, AND S.D. LERNER

Abstract. We present the Transfer of Ownership Protocol (TOOP), a cross-chain protocol that enables the transfer of Bitcoin UTXOs to addresses that are undetermined at locking time. TOOP addresses a fundamental limitation of existing BitVM-like protocols and UTXO blockchains at large, where unlocking transfers are restricted to addresses known and preregistered during setup. By eliminating the financially costly, regulatory problematic, and congestion-prone front-and-reimburse paradigm, TOOP transfers cryptographic capability rather than moving funds through a custodial intermediary: the final owner reconstructs the secret key for the locked UTXO directly from encrypted operator shares.

The protocol operates under a 1-out-of-n honest operator assumption. During the lock phase, an asset is secured under a committee-controlled MuSig2 aggregate key on Bitcoin. During the unlock phase, the legitimate recipient burns the corresponding wrapped asset on a secondary blockchain, receives encrypted key shares via ECIES, selects a valid operator subset, and reconstructs the aggregate secret key locally. An optimistic verification engine on Bitcoin (instantiated with BitVMX) enforces that the unlock is authorized only upon proof of a valid burn and owner-committed group selection.

We set the security of TOOP through a Universally Composable (UC) analysis in the framework of Canetti. We define five ideal functionalities ($\mathcal{F}_{\mathrm{BC}_1}$, $\mathcal{F}_{\mathrm{BC}_2}$, $\mathcal{F}_{\mathrm{MuSig}}$, $\mathcal{F}_{\mathrm{ECIES}}$, $\mathcal{F}_{\mathrm{BitVMX}}$) and a top-level ideal functionality $\mathcal{F}_{\mathrm{TOOP}}$ that captures all security guarantees (asset validity, transfer authenticity, double-spend prevention, ownership transfer security, key share confidentiality, and liveness) in a single object with six formally verified invariants. We construct a simulator and prove, via a hybrid argument based on the IND-CPA security of ECIES, that the protocol UC-realizes $\mathcal{F}_{\mathrm{TOOP}}$ in the hybrid world against static adversaries corrupting up to $n-1$ operators. The UC composition theorem then guarantees that these properties are preserved under arbitrary concurrent composition. We further resolve the relationship between operator key shares and MuSig2 aggregation coefficients through a fixed-reference-set construction, and prove subset reconstruction security under the discrete logarithm assumption.

Keywords. Bitcoin, BitVMX, UC security, cross-chain interoperability.

Contents

1. Introduction

Blockchain interoperability is a concept that has become increasingly important for the widespread adoption and practical utility of blockchain technology. Interoperability means that several blockchains can interact securely and consistently while avoiding new strong trust assumptions.

1.1. **Fundamental approaches to cross-chain technology.** Several methods to achieve cross-chain interoperability can be found in the literature. The interested reader can find in [24] a rigorous treatment of the technology and in SoK by Augusto et al. [1] a treatment of the security and privacy implication of those methods. Here we briefly present these results for completeness.
We model a cross-chain exchange between blockchains (or ledgers) BC_1 and BC_2. Assume entities P and Q operate on blockchains BC_1 and BC_2 respectively. P and Q may be single parties, federations, or even the same party. Our model presents a cross-chain transfer as a four-steps protocol:

(1) *Setup.* The parties exchange information about the involved blockchains BC_1 and BC_2. This includes verification and agreement schemes, asset details and time constraints.
(2) *Locking transaction on* BC_1. The party P broadcasts a *locking* transaction to blockchain BC_1[1]. The transaction is finalized through its consensus protocol. This transaction acts as a *commitment* on chain BC_1
(3) *Verify.* Party Q verifies the commitment on BC_1 using the agreed-upon verification scheme. Based on the result, the protocol either proceeds to commit on BC_2 or aborts.

4a. *Unlocking transaction on* BC_2. Upon successful verification, the party Q sends an *unlocking* transaction to blockchain BC_2.
4b. *Abort.* If verification fails or Q cannot complete the process on BC_2, the protocol aborts by reverting the effects on BC_1, typically through a compensating transaction.

In practice, the entity Q may serve either as a direct beneficiary of the exchange or as a facilitator. In the latter role, Q can be a single entity or a collection of entities. We use the term group in a broad sense, encompassing configurations with heterogeneous governance structures.[2] This distinction gives rise to a variety of implementation strategies, each with its own trade-offs. For clarity, we categorize these strategies based on the number of independent entities involved, distinguishing between cases where Q is a *Single* party and those where it is a *Group*.

1.1.1. *Single* Q. This is the easiest and older approach. In general, P is a user aiming to exchange funds, while Q can either be another user with the opposite goal or a facilitator. We now see both cases.

[1]Such a transaction is said to *lock* the funds of P on chain BC_1, preventing P from spending them. Similarly, to *unlock* funds from party Q means that Q can now use them as it pleases.

[2]Specifically, we consider a company (or user) operating multiple nodes as a single entity, since control is centralized. In contrast, a committee (or federation), where each node is governed by a distinct company or user, is treated as a group.

Atomic Swaps. In the first case, we say that the exchange is *decentralized* since no centralizing process is involved. This kind of exchange is generally called Atomic Swap. Atomic here refers to the atomicity of changes in databases: either both P and Q receive their new asset, or P and Q retain the older one. In particular, a secure atomic swap avoids the case where, for instance, P receives the new asset *and* retains its older one.

The first atomic swap method is based on the Hash-Time Lock contract (HTLC), introduced by Tier Nolan [18]. In an HTLC, after P has found a potential buyer Q, P performs a locking transaction on chain BC_1 by sending a transaction that can be redeemed either by showing the preimage of a hash h and the usual signature (hash-lock) or after some time elapsed and the showing of the signature (time-lock). Notably, Q sends a similar transaction on chain BC_2 using the same hash h.

If P is honest, P shows its preimage on chain BC_2, redeeming Q's transaction before the time has elapsed. If Q is honest, Q uses P's preimage to redeem P's transaction on chain BC_1, completing the exchange.

Conversely, if P is dishonest and does not redeem the transaction on chain BC_2, Q needs to wait for the time to elapse on chain BC_2. Considering the time-value of money[3], this is a serious obstacle.

Company based exchanges. When Q represent a company, we say that the method is *centralized*. In that case, Q's guarantees derive from a legal (or trust based) approach, instead of cryptographic methods. An example of this approach is the company operating the bitcoin wrapped token WBTC [12] on Ethereum.

The advantage for P is in its simplicity. In fact, when Q is a company, P does not have to find the buyer nor perform complex non-standard transaction. In a centralized method, P only sends a transaction on BC_1 redeemable by Q. Furthermore, since guarantees are trust based, Q performs a normal transaction on chain BC_2 too. Here, commitments are trivial.

The tradeoff is that P has to trust both Q and the legal apparatus behind it. For example, if Q does not broadcast a correct transaction on chain BC_2, then P needs to rely on lawyers or Q's customer service to retrieve its funds, since no time-lock mechanism has been set.

1.1.2. *Group Q.* This is the case of *committees* (sometimes called federations). In those cases, Q generally acts as facilitator.

In a committee, Q is a set of n nodes requiring a subset of size t to act together to sign transactions for locking and unlocking funds. An example of a committee is the Liquid Federation [16].

Similarly to companies based exchange, P sends the transaction to the designated address of the committee Q. Then at least t parties need to sign a transaction releasing funds for P on chain BC_2.

Since *only a subset* of the committee is required to act, the probability of successful malicious action is greatly diminished since it requires a collusion of t parties. Clearly, larger t mean a lower probability of success since different companies have different aims at different times.

[3]The time-value of money is the idea that a sum of money is worth more now than the same sum in the future due to its potential earning capacity.

Surprisingly, this reduction of risk does not come with a reduction of usability on the part of P. In fact, P only sends a transaction to an address and the complexity is hidden inside the federation.

1.2. **Current research.** Cross-chain asset transfer protocols must satisfy fundamental security and correctness requirements to ensure reliable interoperability between blockchain networks. We model an asset A as a set whose elements represent the atomic units that can be transferred independently, enabling precise analysis of partial asset transfers and fractional ownership scenarios common in decentralized finance applications.
The research by Sober et al. [21] establishes five essential requirements that any secure cross-chain asset transfer protocol must satisfy. These requirements provide a formal framework for evaluating protocol correctness and security properties.

- R1 Asset ownership verification requires that asset destruction operations respect legitimate ownership boundaries. When a sender attempts to burn an asset subset X on the source blockchain BC_1, the operation should succeed only if $X \subseteq A^{\mathsf{BC}_1}_{\text{Sender}}$, where $A^{\mathsf{BC}_1}_{\text{Sender}}$ represents the assets legitimately controlled by the sender. The burning operation permanently removes assets from circulation on the source blockchain, making them unavailable for future transactions, while enabling their recreation on the destination blockchain.
- R2 Transfer authenticity ensures that asset recreation occurs only after verified destruction on the source blockchain. When transferring asset subset X from source blockchain BC_1 to destination blockchain BC_2, the assets should be recreated on BC_2 only after cryptographic proof demonstrates that X has been legitimately burned on BC_1. This requirement prevents counterfeit asset creation and maintains the fundamental invariant that cross-chain transfers neither create nor destroy value.
- R3 Double-spending prevention mandates that burned assets can be recreated exactly once across all destination blockchains. If an asset subset X is burned on one blockchain, it can be recreated on at most one other blockchain, preventing scenarios where the same assets appear simultaneously on multiple chains and violate conservation of value.
- R4 Guaranteed recreation establishes that burned assets will eventually be recreated on the intended destination blockchain within a bounded timeframe. This liveness property ensures that legitimate transfers do not result in permanent asset loss due to protocol failures or network partitions.
- R5 Confirmation feedback provides optional verification that asset recreation has completed successfully on the destination blockchain. In this case, the source blockchain should eventually receive confirmation that the asset subset X has been successfully recreated, enabling applications that require end-to-end transfer verification. This requirement supports decentralized finality, allowing any network participant to submit confirmation proofs rather than relying on trusted intermediaries.

XClaim [23] implements cross-blockchain transfers through designated vault operators, which lock assets on the source blockchain and issue corresponding wrapped tokens on the destination blockchain. In this system, vault operators provide overcollateralization in the destination blockchain's native currency to guarantee the value

of issued tokens. When users request cross-chain transfers, they deposit assets with a vault operator on the source blockchain, and the vault operator mints equivalent wrapped tokens on the destination blockchain after the deposit is confirmed.

The protocol makes use of blockchain relays to verify locking transactions on the source blockchain, enabling automatic verification of asset deposits without requiring trusted oracles. This relay-based verification mechanism satisfies R1 by ensuring that only legitimately deposited assets trigger wrapped token issuance on the destination blockchain.

XClaim's reliance on individual vault operators for transfer execution creates significant centralization risks that violate fundamental decentralization requirements. Each transfer depends on a single vault operator who controls both the locked assets and the minting process for wrapped tokens. This centralized control structure fails to prevent double-spending scenarios where malicious vault operators could potentially issue wrapped tokens without proper asset backing or refuse to process legitimate redemption requests. The protocol also lacks guaranteed recreation mechanisms because vault operators can unilaterally decide whether to complete transfer operations, causing permanent asset loss if operators become unavailable or act maliciously.

This centralized vault model contrasts sharply with distributed approaches where multiple independent parties participate in transfer finalization, reducing single points of failure and improving protocol resilience. The concentration of control in individual vault operators fundamentally limits XClaim's ability to provide the trustless guarantees expected from decentralized cross-chain protocols.

Karantias et al. [11] provide a cryptographic treatment of proof-of-burn mechanisms, establishing rigorous foundations for protocols that enable verifiable cryptocurrency destruction. While proof-of-burn has been utilized in various blockchain applications, previous implementations lacked formal security analysis and standardized definitions, limiting their theoretical understanding and practical deployment.

The research introduces a comprehensive cryptographic framework consisting of two fundamental functions that together constitute a complete burn protocol. The GenBurnAddr function generates cryptocurrency addresses with the critical property that any funds sent to these addresses become permanently and irrevocably destroyed. The BurnVerify function provides cryptographic verification that a given address represents a legitimate burn address, enabling independent parties to confirm the validity of burn operations without requiring trusted intermediaries.

The formal analysis establishes three security properties that define robust burn protocols. Unspendability ensures that addresses verified as burn addresses cannot be used for spending operations, providing mathematical guarantees that burned funds remain permanently inaccessible. Binding enables the association of arbitrary metadata with specific burn operations, allowing burn addresses to encode information such as destination blockchain identifiers or transfer parameters. Uncensorability mandates that burn addresses remain computationally indistinguishable from regular cryptocurrency addresses, preventing censorship attempts that might block burn transactions based on address analysis.

The protocol design achieves broad compatibility with existing cryptocurrency systems through its construction simplicity and flexibility. The scheme operates effectively across all popular cryptocurrencies without requiring protocol modifications or

specialized wallet software, enabling users to perform burn operations through standard transaction mechanisms. Security analysis demonstrates protocol correctness under the Random Oracle model, providing formal guarantees for the cryptographic constructions.

Although satisfying R1, the protocol does not address R3 and R5, lacking decentralized finality and transfer confirmation mechanisms. Pillai et al. [19] introduce a burn-to-claim protocol which requires specialized gateway nodes for cross-network mining and burn transaction verification. While meeting R1, the protocol fails to address R3, R4, and R5, lacking decentralized finality, true decentralization, and transfer confirmations.

AucSwap [15] models cross-chain transfers as Vickery auctions, using HTLCs for asset exchange. Although this approach satisfies R1 and R2, it functions more as an asset exchange system than a transfer protocol and does not fully address R3, R4, and R5.

Zendoo [9] uses zero-knowledge proofs for transaction verification (R1) but does not address requirements R3, R4, and R5. Its specific sidechain architecture requirements limit implementation potential.

Van Glabbeek [10] suggests a protocol that adapts multi-hop payment channel concepts, emphasizing finality (R3). While satisfying R2, implementation details remain underspecified for R4 and R5.

DeXTT [5] synchronizes the existence of assets across blockchains. Its claim-first transaction model violates R2, and the protocol lacks transfer confirmations (R5), although it partially addresses R1 and R4 through its decentralized synchronization process.

Polkadot [22] implements cross-chain Message Passing (XCMP) using Merkle tree-based queuing. Cosmos employs the TCP/IP-inspired Interblockchain Communication (IBC) protocol. While both platforms address R1 and R2, their specialized blockchain requirements impact R4, and they lack specific implementations for R3 and R5.

The Ownership Transfer and Execution (OTEx) system [7] facilitates asset ownership transfer without asset movement, comprising:

(1) Inter Blockchain Communication (IBC) protocol for message formatting.
(2) IBC Gateways for smart contract execution and ownership state transfers.
(3) Auxiliary blockchain logging for transaction traceability.

While addressing R1 and R2 through its comprehensive logging and gateway system, the reliance of the protocol on specialized gateways affects R4.

1.3. **Contributions of this research.** Our work advances cross-chain asset transfer protocols through several key innovations that address critical limitations in existing systems. Building upon the requirements established by Sober et al., our research introduces a proposal for a bidirectional ownership transfer protocol with dynamic address support. Most importantly, the proposed Transfer of Ownership Protocol (TOOP) is, to our knowledge, the first protocol to enable locking an UTXO towards an address that is undetermined at the time of locking. The usefulness of this functionality is mainly showcased in our work as a core part of cross-chain transfers in the Cardinal project, bridging Bitcoin Ordinals to the Cardano blockchain.

This feature distinguishes our protocol from existing cross-chain systems such as DeXTT and OTEx, which require recipients to be specified and registered during

the initial asset locking phase. Our approach enables ownership updates to arbitrary legitimate addresses after the lock phase has completed, providing operational flexibility that addresses a limitation in BitVM-based cross-chain protocols.

The core insight underlying TOOP is that transferring ownership of a Bitcoin UTXO is equivalent to transferring the ability to reconstruct its private key. Traditional bridges move funds through a custodial intermediary: the user deposits to a bridge-controlled address, and the bridge later releases funds to the recipient. In contrast, TOOP transfers cryptographic capability. The locked UTXO remains at a fixed address throughout its lifecycle; what changes is who can reconstruct the secret key needed to spend it. This key-transfer paradigm eliminates the custodial step entirely: operators hold key shares but cannot unilaterally spend, and the final owner reconstructs the aggregate secret only at redemption time.

The key-transfer paradigm offers advantages beyond security. First, capital efficiency: because operators hold key shares rather than custodying funds, TOOP does not require the overcollateralization typical of vault-based bridges. Operators post bonds to ensure protocol compliance, but these bonds need not scale with the total value locked. Second, regulatory clarity: operators are key-share holders, not custodians. User funds are never "held" by the bridge in the traditional sense—they remain locked at a Bitcoin address that no single party controls. This distinction may simplify the regulatory posture of TOOP-based bridges compared to custodial alternatives.

2. Preliminaries

2.1. **Notation.** We establish the notation which will be used along this paper. Let BC_1 and BC_2 be two blockchain systems. In our setting BC_1 will be Bitcoin. Let $\mathcal{O} = \{O_1, \ldots, O_n\}$ be the set of operators, each modelled as probabilistic polynomial-time (PPT) machines. Let $\mathcal{A}$ denote an adversary, also modelled as a PPT machine. We denote by ID the power set of operator subsets, and by $\mathcal{U}$ denote the universe of all possible assets. Let κ denote the security parameter. All algorithms and security experiments receive 1^κ as implicit input.

2.2. **Security definitions and protocol properties.**

Definition 2.1 (Owner)**.** Let us consider $\mathcal{BC}$ be a blockchain and U be the universe of all digital assets on $\mathcal{BC}$. An entity *Owner* is the owner of asset subset $X \subseteq \mathcal{U}$ if and only if *Owner* possesses, for each asset $x \in X$, the private key material sk_x corresponding to the public key pk_x that controls the spending conditions of x.

Definition 2.2 (Asset validity)**.** A protocol Π satisfies asset validity if, for any asset subset $X \subseteq \mathsf{U}$ and any Θ_0, the probability that a lock transaction $Lock(X, \Theta_0)$ is accepted on BC_1 when $X \nsubseteq Assets(\Theta_0, \mathsf{BC}_1)$ is negligible in κ:

$$\Pr\left[\begin{array}{l} \text{params} \leftarrow \text{Setup}(1^\kappa); (X, \Theta_0) \leftarrow \mathcal{A}(\text{params}); \\ X \nsubseteq \text{Assets}(\Theta_0, \mathsf{BC}_1); \\ \text{Lock}(X, \Theta_0) \text{ accepted on } \mathsf{BC}_1 \end{array}\right] \leq \text{negl}(\kappa)$$

Here, $Assets(Owner, \mathsf{BC})$ is the set of all assets that are legitimately owned and controllable by *Owner* on BC at the time of evaluation, and the function $Lock(X, Owner)$ represents a transaction where *Owner* transfers asset subset X to the operators' aggregate multi-signature address.

Definition 2.3 (Transfer authenticity)**.** A protocol Π satisfies transfer authenticity if, for any asset subset $X \subseteq \mathsf{U}$, the probability that a recreation transaction $Create(X, \mathsf{BC}_2)$ is accepted without a corresponding valid $Burn(X, \mathsf{BC}_1)$ having occurred is negligible in κ:

$$\Pr\left[\begin{array}{l}\text{params} \leftarrow \text{Setup}(1^\kappa); X \leftarrow \mathcal{A}(\text{params});\\ \text{Create}(X, \mathsf{BC}_2) \text{ accepted};\\ \nexists \text{ valid Burn}(X, \mathsf{BC}_1)\end{array}\right] \leq \text{negl}(\kappa)$$

Here, $Create(X, \mathsf{BC}_2)$ represents the minting of a wrapped asset X on the secondary blockchain BC_2. Furthermore, $Burn(X, \Theta_1)$ represents a transaction on BC_2 where the wrapped asset X is burnt.

Definition 2.4 (Double-spend prevention)**.** A protocol Π satisfies double-spend prevention if, for any asset subset $X \subseteq \mathsf{U}$ that has been burnt in BC_1, the probability that X is successfully recreated more than once is negligible in κ:

$$\Pr\left[\begin{array}{l}\text{params} \leftarrow \text{Setup}(1^\kappa); X \leftarrow \mathcal{A}(\text{params});\\ \text{Burn}(X, \mathsf{BC}_1) \text{ accepted};\\ \text{Create}(X, \mathsf{BC}_2) \text{ accepted};\\ \text{Create}'(X, \mathsf{BC}_2') \text{ accepted}\end{array}\right] \leq \text{negl}(\kappa)$$

where $Create'$ is a second creation operation, possibly on a different blockchain BC_2'. Here, since $\mathsf{BC}_2' = \mathsf{BC}_2$, the definition reduces to: each locked asset can be minted at most once on BC_2.

Remark 2.5. Definitions 2.2-2.9 follow the generic terminology of [21], where *Burn* denotes asset destruction on the source chain and *Create* denotes asset creation on the destination chain. In our instantiation, the source-chain operation is a *Lock* (on BC_1 = Bitcoin) and the destination-chain operation is a *Mint* (on BC_2). The reverse direction uses *Burn* on BC_2 and *Unlock* on BC_1. We adopt the generic names in the definitions for consistency with prior work; the mapping to TOOP-specific operations is made explicit in Section 3.

Definition 2.6 (Ownership transfer security)**.** A protocol Π satisfies ownership transfer security if for any asset subset $X \subseteq \mathsf{U}$ initially owned by Θ_0 on BC_1 and any PPT adversary $\mathcal{A}$, writing $\mathcal{G}(\Theta_1, X)$ for the event that assets X are transferred to an address under the control of Θ_1:

$$\Pr\left[\begin{array}{l}params \leftarrow \mathsf{Setup}(1^\kappa); (\mathsf{sk}_{\Theta_1}, \mathsf{pk}_{\Theta_1}) \leftarrow \mathsf{KeyGen}(1^\kappa);\\ (X, \Theta_0) \leftarrow \mathcal{A}(params, \mathsf{pk}_{\Theta_1});\\ \mathsf{Lock}(X, \Theta_0) \text{ accepted on } \mathsf{BC}_1;\\ \mathsf{Burn}(X, \mathsf{pk}_{\Theta_1}) \text{ accepted on } \mathsf{BC}_2;\\ \neg\mathcal{G}(\Theta_1, X)\end{array}\right] \leq \mathsf{negl}(\kappa)$$

Remark 2.7. Here pk_{Θ_1} denotes the fresh ECIES public key generated by Θ_1 for this transfer session (written $\widetilde{\mathsf{pk}}_1$ in the protocol description of Section 3.2), not Θ_1's long-term identity key.

Remark 2.8. Here *under the control of* Θ_1 means Θ_1 possesses the spending key for the address. The adversary $\mathcal{A}$ may corrupt up to $n-1$ operators and interact with the protocol during execution.

Definition 2.9 (Liveness)**.** A protocol Π satisfies liveness if, for any subset $X \subseteq \mathsf{U}$ locked by Θ_0 and burned Θ_1 with a valid public key, and assuming at least one honest operator follow the protocol, the assets X will eventually be transferred to Θ_1 on BC_1. If we denote Δ a time bound specified by the system then, given at least one honest operator:

$$\Pr\left[\begin{array}{l} \text{params} \leftarrow \text{Setup}(1^\kappa); \\ \text{Lock}(X, \Theta_0) \text{ accepted at time } t_0; \\ \text{Burn}(X, \mathsf{pk}_{\Theta_1}) \text{ accepted at time } t_1; \\ \exists t_2 \leq t_1 + \Delta : \mathcal{G}(\Theta_1, X) \text{ at time } t_2 \end{array}\right] \geq 1 - \text{negl}(\kappa)$$

Definition 2.10 (1-out-of-n security)**.** A protocol Π satisfies 1-out-of-n security if for any adversary $\mathcal{A}$ that can corrupt up to $n-1$ operators, the protocol maintains asset validity, transfer authenticity, double-spend prevention, and ownership transfer security, with overwhelming probability. That is, for any security property $\mathcal{P}$ defined above and any adversary $\mathcal{A}$ controlling a subset $\mathcal{C} \subset \mathcal{O}$ such that $|\mathcal{C}| \leq n-1$:

$$\Pr\left[\begin{array}{l} \text{params} \leftarrow \text{Setup}(1^\kappa); \\ \mathcal{C} \leftarrow \mathcal{A}(\text{params}) \text{ where } |\mathcal{C}| \leq n-1; \\ \mathcal{A} \text{ violates property } \mathcal{P} \end{array}\right] \leq \text{negl}(\kappa)$$

Definition 2.11 (Key share confidentiality)**.** A protocol Π satisfies key share confidentiality if, for any adversary $\mathcal{A}$ that does not know Θ_1 private key, the probability that $\mathcal{A}$ can extract any operator's key share from the encrypted communications is negligible in κ. Formally, for any operator O_i and its key share k_i:

$$\Pr\left[\begin{array}{l} \text{params} \leftarrow \text{Setup}(1^\kappa); \\ (\mathsf{pk}_{\Theta_1}) \leftarrow \mathcal{A}(\text{params}); \\ \text{encShare}_i \leftarrow \text{ECIES.Encrypt}(\mathsf{pk}_{\Theta_1}, k_i); \\ k_i' \leftarrow \mathcal{A}(\text{encShare}_i); \\ k_i' = k_i \end{array}\right] \leq \text{negl}(\kappa)$$

Remark 2.12. All security properties above are formulated under the static corruption model: the adversary's corruption choices are fixed before protocol execution begins.

2.3. **Dependencies with respect to the 1-out-of-n assumption.** The 1-out-of-n honesty assumption is basic in this proposal. This assumption fundamentally shapes how the protocol achieves its security guarantees and determines which properties can be maintained even under adversarial conditions. The following analysis examines each security property and clarifies its relationship to this assumption. We consider 3 categories:

(1) Critical dependency:
   (a) Asset validity: on BC_1 (Bitcoin), asset validity is enforced at the consensus level by the UTXO model and Schnorr unforgeability (a user cannot lock a UTXO she does not own because the spending signature will be invalid). The 1-out-of-n assumption is critical for asset validity *on* BC_2: it ensures that wrapped supply inflation cannot occur without a corresponding valid lock, since the honest operator will refuse to participate in (or will challenge) any minting not backed by a confirmed lock.

(b) Transfer authenticity depends fundamentally on honest operator participation during the unlocking phase. When assets are burned on the secondary blockchain, operators must verify this burn transaction before providing their encrypted key shares. The honest operator ensures that assets can only be unlocked on Bitcoin when a corresponding legitimate burn has occurred on the secondary chain. Without this verification by at least one honest participant, the protocol could not maintain the crucial invariant that prevents unauthorized asset recreation.
(c) Double-spend prevention requires honest operators to maintain consistent protocol state and refuse participation in multiple unlocking operations for the same asset. The honest operator acts as a safeguard against attempts by corrupted operators to facilitate multiple unlocks of the same locked asset. This property cannot be maintained without the honest operator's disciplined adherence to the protocol's state management requirements.
(d) Liveness depends on the 1-out-of-n assumption to ensure that legitimate transfer requests eventually complete. The honest operator guarantees that valid lock and burn transactions will be processed, preventing corrupted operators from indefinitely blocking legitimate transfers. Without at least one honest operator willing to participate in the protocol, the system could deadlock, preventing any asset transfers from completing.
(e) Secure group selection relies on honest operators to provide valid encrypted key shares to legitimate recipients. The protocol's security depends on Θ_1 being able to select a group that includes at least one honest operator. The honest operator ensures that valid key shares are always available to legitimate recipients, preventing scenarios where only corrupted operators respond to transfer requests.

(2) Partial dependency: Ownership transfer security has a nuanced relationship with the 1-out-of-n assumption. While the ECIES encryption used for key share distribution provides confidentiality regardless of operator honesty, the overall ownership transfer security depends on honest operators to properly verify recipients and refuse to provide key shares to unauthorized parties. The honest operator ensures that the protocol's ownership verification mechanisms function correctly.
(3) Independent of the 1-out-of-n honesty assumption: Key share confidentiality represents the only security property that does not directly depend on the 1-out-of-n assumption. This property is guaranteed by the cryptographic security of ECIES encryption, which prevents adversaries from extracting key shares without possession of the recipient's private key. The confidentiality holds regardless of how many operators are corrupted, as it relies solely on the underlying cryptographic assumptions rather than honest behaviour.

The 1-out-of-n honesty assumption is not just a convenience but an essential requirement for nearly all security properties of the protocol. The assumption enables a trust-minimized approach that avoids the need for honest majorities while still providing strong security guarantees. The protocol's architecture leverages this assumption efficiently by ensuring that a single honest operator can prevent all major classes of attacks while enabling legitimate operations.

2.4. **Threat model.** We present a threat model for the formal characterization of potential adversaries, their capabilities, and the attack vectors they might exploit. The model below considers adversaries who can corrupt a subset of protocol participants, manipulate network communications, and exploit timing differences between blockchain confirmations. We focus on the 1-out-of-n honest operator assumption, which distinguishes our approach from traditional multi-signature schemes. The model of this protocol is given by the following entities and participants:
We assume the source blockchain BC_1 to be Bitcoin. Let us assume that BC_2 is the destination blockchain: A smart contract-enabled blockchain that can execute arbitrary logic
The protocol involves four key participants. Operators $\mathcal{O} = \{O_1, O_2, \ldots, O_n\}$ are entities responsible for facilitating cross-chain transfers. Θ_0 is the initial asset owner on BC_1, while Θ_1 is the recipient of the asset after cross-chain transfer. The adversary $\mathcal{A}$ represents any entity attempting to compromise the protocol.
The protocol consists of four main technical components. The BitVMX Program executes the verification logic for unlocking assets on Bitcoin. The Smart Contract manages wrapped asset creation and burning on BC_2. MuSig2 Aggregation enables multi-signature creation among operators, while ECIES Encryption secures communication between operators and owners.
The protocol relies on five trust assumptions, namely: the 1-out-of-n honest operator assumption requires that at least one operator among n follows the protocol correctly, serving as the fundamental trust assumption. Both BC_1 and BC_2 operate correctly, maintain consensus, and execute transactions as programmed. BitVMX security assumes the verification system operates according to its specifications and correctly enables challenge-response protocols. Non-collusion requires that owners and operators maintain independence, with no universal collusion between all participants. Finally, key management assumes that participants securely generate and store their cryptographic keys.
The adversary model defines five core capabilities that establish the security boundaries of the protocol. The adversary can achieve operator corruption by compromising up to $n - 1$ operators, gaining complete access to their private keys and the ability to make them deviate arbitrarily from the established protocol. Through network observation capabilities, the adversary can monitor all public network communications, including comprehensive visibility into all transactions occurring on both blockchain networks. The adversary possesses network delay capabilities, allowing it to delay network messages and blockchain transactions up to a specified bound Δ, though it cannot indefinitely block these communications. Additionally, the adversary can execute transaction creation by generating arbitrary valid transactions on both blockchain networks. The model constrains the adversary through computational bounds, defining it as a probabilistic polynomial-time (PPT) machine that operates within established computational limitations.
On the other hand, the adversary is explicitly unable to corrupt all n operators simultaneously, break the underlying cryptographic primitives, violate the consensus rules of either blockchain, indefinitely prevent transactions from being confirmed, and access encrypted communications without the corresponding private key.
The protocol operates within the following structured communication framework. Public blockchain communication ensures that all blockchain transactions remain

publicly visible and immutable once confirmed, providing transparency and verifiability for all network participants. Private operator-owner communication utilizes encrypted channels through ECIES encryption, protecting sensitive information exchanges between these critical parties. The system operates under Δ-synchronous communication: every message sent by an honest party is delivered to all recipients within time Δ. Despite variable actual delays, the known upper bound Δ enables the protocol to set deterministic timeouts. Despite these delays, reliable delivery guarantees ensure that messages and transactions are eventually delivered within the established time bound Δ. The framework explicitly operates without trusted broadcast capabilities, requiring that all multi-party communication be secured through cryptographic means rather than relying on trusted intermediaries.

Remark 2.13. We note that the Δ-synchrony assumption is not formalized via a clock functionality. The liveness guarantee (Invariant I6 of $\mathcal{F}_{\text{TOOP}}$) is conditional on this synchrony assumption being enforced by the network environment. A full treatment with $\mathcal{G}_{\mathsf{clock}}$ is deferred to future work. As a consequence, the liveness guarantee (I6) does not compose with clock-dependent protocols without additional argument

The security model relies on the following cryptographic assumptions. The discrete logarithm problem (DLP) establishes that given $Y = x \cdot G$ where $G \in E(\mathbb{Z}_q)$ is a generator, determining the scalar x remains computationally infeasible for adversaries. Building upon this foundation, the Elliptic Curve Diffie-Hellman problem assumes that when provided with $a \cdot G$ and $b \cdot G$ for unknown scalars $a, b \in \mathbb{Z}_q$, computing $a \cdot b \cdot G$ presents an insurmountable computational challenge. The elliptic curve discrete logarithm problem (ECDLP) represents the specific instantiation of the DLP within elliptic curve groups $E(\mathbb{Z}_q)$, providing the cryptographic hardness for elliptic curve-based operations. The analysis employs the random oracle model, treating hash functions as perfect random oracles for security evaluation purposes. The encryption components rely on IND-CPA security, ensuring that the symmetric encryption component of ECIES maintains indistinguishability under chosen-plaintext attacks. Finally, the framework requires existential unforgeability, guaranteeing that the digital signature schemes employed remain existentially unforgeable under chosen-message attacks. This threat model addresses the following categories of attacks:

(1) Protocol flow attacks: The protocol must defend against several categories of attacks that target the execution flow and timing aspects of cross-chain operations. Replay attacks represent a fundamental threat, where adversaries attempt to reuse signatures or protocol messages from previous legitimate transactions to achieve unauthorized asset transfers. Race conditions pose timing-based vulnerabilities that adversaries can exploit by manipulating the sequence or timing of cross-chain events to create inconsistent states or bypass security checks. Griefing attacks involve malicious operators who deliberately attempt to block or delay protocol execution, potentially causing asset freezing or denial of service without necessarily stealing assets. Eclipse attacks present network-level threats where adversaries isolate honest operators from the broader network, preventing them from receiving critical updates or participating in consensus mechanisms.
(2) Cryptographic attacks: The cryptographic foundation of the protocol faces multiple attack vectors that target the underlying mathematical primitives

and their implementations. Key compromise attacks occur when adversaries obtain private keys through side-channel analysis, implementation vulnerabilities, or other technical exploitation methods that bypass the intended cryptographic protections. Signature forgery represents attempts by adversaries to create valid signatures for unauthorized actions without possessing the corresponding private keys, potentially enabling unauthorized asset transfers or protocol manipulations. Rogue key attacks specifically target multi-signature schemes by allowing adversaries to manipulate their public key contributions during the aggregation process, potentially gaining disproportionate influence over collective signature operations.

(3) Asset-specific attacks: The protocol must address attacks that directly target asset integrity and ownership throughout the cross-chain transfer process. Double-spending attacks represent attempts by adversaries to unlock or transfer the same asset multiple times, potentially creating artificial inflation or theft scenarios across the participating blockchain networks. Unauthorized transfer attacks involve adversaries attempting to redirect legitimate asset transfers to unauthorized recipients, effectively stealing assets during the cross-chain migration process. Asset freezing attacks occur when malicious operators deliberately prevent asset unlocking or completion of transfers, potentially resulting in permanent loss of access to legitimate user assets.

(4) Cross-chain consistency attacks: The inherent complexity of managing state across multiple blockchain networks creates opportunities for attacks that exploit inconsistencies between chains. Invalid state transition attacks involve adversaries creating or exploiting inconsistent states between the source and destination blockchains, potentially allowing unauthorized asset creation or destruction. Finality differences present attack vectors where adversaries exploit the varying finality guarantees and confirmation times between different blockchain networks to create temporary or permanent inconsistencies. Chain reorganization attacks leverage the possibility of blockchain reorganizations or forks to undo completed asset transfers, potentially enabling sophisticated double-spending scenarios that span multiple blockchain networks.

In particular, this model considers the following specific threat scenarios:

(1) Operator collusion: The most significant threat to protocol security appears when up to $n-1$ operators coordinate to compromise the system through coordinated malicious behaviour. These collusive operators can pursue multiple attack vectors designed to subvert the protocol's intended functionality and security guarantees. They may attempt to unlock assets without corresponding legitimate burn transactions on the secondary blockchain, effectively creating unauthorized asset transfers that bypass the protocol's cross-chain verification mechanisms. Additionally, colluding operators can refuse to process legitimate unlock requests from honest users, creating denial-of-service conditions that prevent rightful asset access. The provision of incorrect key shares to legitimate recipients represents another attack vector, where operators deliberately distribute invalid cryptographic material to prevent successful asset recovery. Furthermore, malicious operators may manipulate the BitVMX execution environment to favour their coordinated interests, potentially compromising the

verification logic that ensures protocol correctness. To address these threats, the protocol must maintain security even when facing $n - 1$ colluding operators, ensuring that at least one honest operator retains the capability to prevent unauthorized actions. The BitVMX challenge-response mechanism serves as a critical security requirement, enabling the detection of invalid executions and maintaining protocol integrity despite widespread operator collusion.

(2) Owner impersonation: Impersonation attacks employ various technical approaches to compromise the recipient identification and verification mechanisms within the protocol. Adversaries may attempt to intercept encrypted key shares that operators transmit to the intended Θ_1, positioning themselves to capture sensitive cryptographic material during the communication process. Once intercepted, attackers seek to decrypt these key shares without possessing Θ_1 legitimate private key, requiring them to break the ECIES encryption or exploit implementation vulnerabilities. Another attack vector involves creating forged proofs of ownership on the secondary blockchain, where adversaries attempt to present false evidence of their authority to receive cross-chain transferred assets. To counter these threats, the protocol must implement robust cryptographic verification of Θ_1 identity, ensuring that only legitimate recipients can access transferred assets. Key shares must maintain strict confidentiality, remaining accessible exclusively to the intended Θ_1 throughout the entire transfer process. Additionally, the protocol must verify that burn transactions originate from legitimate asset owners, preventing unauthorized parties from initiating transfers using assets they do not rightfully control.

(3) Cross-chain exploitation: Coordinating operations across multiple blockchain networks creates opportunities for adversaries to exploit inherent differences and inconsistencies between the participating chains. These exploitation attacks target the complex synchronization requirements and varying operational characteristics of different blockchain systems. Adversaries may attempt to create inconsistent states between the source and destination chains, leveraging timing differences or operational discrepancies to establish contradictory asset states that benefit their malicious objectives. Finality differences between blockchain networks present particularly sophisticated attack opportunities, where adversaries exploit varying confirmation requirements and finality guarantees to revert transactions on one chain after achieving completion on another. This temporal manipulation can enable complex double-spending scenarios that span multiple networks. Additionally, adversaries may manipulate timestamps or block confirmations to create artificial timing conditions that bypass the protocol's synchronization safeguards. To address these cross-chain vulnerabilities, the protocol must implement consistent state verification mechanisms that ensure coherent asset states across all participating blockchain networks. Sufficient confirmation waiting periods serve as essential security requirements, providing adequate time for transaction finality to prevent exploitation of timing differences. The protocol must also enforce atomic execution of cross-chain operations, ensuring that asset transfers either complete successfully across all involved chains or fail entirely, preventing partial states that adversaries could exploit.

2.5. **Blockchain interoperability.** One of the most important challenges in the blockchain ecosystem has been the complexities and insecurity of communication and asset transfer between different networks. Blockchain bridges solve this problem by connecting two or more blockchain networks, enabling message passing and cross-chain transactions, enabling flawless interoperability
The basic process involves locking an asset on one blockchain and representing it on another through a wrapped token. Smart contracts handle the transaction, ensuring accuracy and security throughout the process:

(1) A user locks an asset on the source blockchain.
(2) A wrapped token, representing the locked asset, is minted on the destination blockchain.
(3) Smart contracts manage this process, ensuring that transactions are valid and secure.

2.6. **Types of blockchain bridge.** Bridging blockchains protocols, each with its own methods and trust requirements, can be categorized into trusted, trust-minimized and trustless models.

(1) Trusted bridges: trusted bridges rely on a central entity or a group of validators to oversee operations and ensure security.
    (a) Custodial bridges: A centralized entity holds the original assets and mints their equivalents on the destination chain.
    (b) Federated bridges: These are operated by a group of validators who manage assets and verify transactions. We find two models:
        (i) m – of – n: requires a qualified majority (m out of n, typically $m > n/2$) to approve operations.
        (ii) 1 – of – n: requires just one honest validator to process a transaction.
(2) Trustless bridges: trustless bridges eliminate the need for centralized entities and replace them with cryptographic schemes and smart contracts for transaction validation. The essential functioning follows:
    (a) A user deposits an asset into a smart contract on the source chain, which locks it, making it unusable until the process completes.
    (b) The contract generates a proof of both the deposit and the lock.
    (c) A relayer (or oracle) forwards the proof to a smart contract on the destination chain.
    (d) The destination contract verifies the proof and mints the equivalent asset.
    (e) Upon successful minting confirmation, the locked asset in the source chain is burned, ensuring that no duplicate assets exist across chains.
(3) Trust-Minimized Bridges: Trustless bridges cannot be built for the Bitcoin blockchain given today's technology. Trusted bridges, either custodial or federated, require a level of centralization that is not acceptable for many Bitcoin use cases.

    Trust-minimizing bridges achieve a balanced solution in which a group of operators oversees the management of the bridge, while a single honest one can prevent invalid operations from being accepted.

2.6.1. *Key components.* Blockchain bridges are powered by several components that play distinct roles in ensuring secure and efficient operations.

(1) Custodians: manage locked assets in centralized bridges and handle deposits and withdrawals.
(2) Oracles: feed external data into blockchains and monitor activity, such as prices or contract events.
(3) Validators: verify transactions, enforce rules, and confirm deposits before minting wrapped assets.

Each component contributes to the functionality and reliability of the bridge, whether centralized or decentralized.

2.6.2. *Challenges in using bridges.* While blockchain bridges offer powerful interoperability solutions, they face notable challenges that must be addressed for widespread adoption:

(1) Security: bridges are high-value targets for attackers. Trustless bridges, while decentralized, can be exploited due to their complexity. Regular audits and layered security are essential.
(2) Technical complexity: building bridges for multiple networks requires navigating diverse protocols and architectures.
(3) Regulation: operating across jurisdictions with varying laws makes compliance challenging.
(4) Scalability: managing large transaction volumes and optimizing costs while maintaining efficiency can strain bridge operations.

2.6.3. *Bitcoin bridges.* The development of Bitcoin bridges offers particular challenges, indeed: Bitcoin's scripting limitations create fundamental constraints for cross-chain interoperability protocols. The UTXO model's inherent restrictions on state management, combined with Bitcoin Script's limited expressiveness, shape the design space for bridging solutions. The absence of covenant support in Bitcoin presents a challenge, since funds cannot be programmatically constrained to specific destination addresses based on external validation conditions. This limitation forces bridges to implement multi-party coordination protocols rather than direct programmatic constraints.

State management represents another significant challenge. Without the ability to maintain mutable state within transactions, bridges must implement complex UTXO management schemes to track cross-chain assets and their state. This creates additional complexity in ensuring atomicity and preventing replay attacks across bridge operations.

The computational constraints of Bitcoin Script further restrict the types of validation logic that can be performed on-chain. Complex operations like Merkle proof verification or dynamic validator set management cannot be implemented directly within Bitcoin's scripting system.

These constraints have led to several architectural patterns in the ecosystem.

(1) Federation-based approaches leverage multi-signature schemes to implement bridge security through distributed trust. While this provides a practical solution, it introduces additional trust assumptions compared to fully programmatic approaches possible on more expressive platforms.
(2) Hash time-locked contracts (HTLCs) represent another pattern that works within Bitcoin's constraints. However, their application is primarily limited

to atomic swap scenarios rather than general-purpose bridging protocols. The inability to implement complex validation logic on-chain means that HTLC-based approaches struggle to scale beyond simple two-party exchanges.

(3) Layer-2 protocols represent an alternative approach, introducing additional programmability through separate security domains. Solutions like Liquid and Rootstock implement more expressive scripting capabilities, enabling complex bridge logic at the cost of additional trust assumptions in their security models.

These architectural patterns demonstrate how Bitcoin's security-focused design decisions continue to influence the development of cross-chain interoperability solutions. The trade-off between trustlessness and functionality remains a central consideration in bridge design, with different approaches optimizing for different points along this spectrum.

2.7. **BitVMX.** BitVMX [13] is a virtual CPU design that allows arbitrary programs to run on Bitcoin through an optimistic verification system. It builds on the challenge-response framework introduced by BitVM [2], implementing a general-purpose CPU that can be verified using Bitcoin's scripting capabilities. The system is compatible with common processor architectures like RISC-V and MIPS.

The key innovation of BitVMX lies in its streamlined approach. It employs hash chains to track program execution, uses memory-mapped registers, and introduces an enhanced challenge-response protocol. The system includes a message linking protocol that enables authenticated communication between participants, effectively creating stateful smart contracts by maintaining shared state across transactions.

The verification process uses presigned transactions to manage challenge-response interactions. When disputes arise, the system leverages the hash chain of program execution to pinpoint and investigate computational errors through n-ary search. This represents a significant improvement over BitVM, as BitVMX eliminates the need for Merkle trees in CPU instructions and memory operations, while also removing dependence on signature equivocations. These improvements reduce complexity and make BitVMX a strong alternative to BitVM2 [14].

In practice, BitVMX operates as a two-party system between a prover (operator) and a verifier, though it can be expanded to accommodate multiple verifiers. When funds are locked in a Bitcoin UTXO, they can only be spent based on the outcome of a predefined program with specific inputs. The operator must demonstrate correct program execution to access the funds. If the verifier agrees with the execution results, the process proceeds smoothly. If not, both parties enter a dispute resolution protocol on the Bitcoin blockchain.

The execution tracking system in BitVMX represents a notable advancement. Both parties execute the program locally, generating both an execution trace and a hash chain for each step. The hash chain is built by combining each step's trace with the previous hash and applying a secure hashing function. This creates an unforgeable record of execution, where any computational differences between parties result in divergent hash chains.

BitVMX's n-ary search capability allows for faster identification of computational conflicts compared to BitVM's binary search approach. Once an error is located, specialized challenges can determine its precise nature, such as tracking memory access

patterns to identify incorrect memory operations. The system's response verification ensures that all responses directly correspond to specific challenges, allowing the challenger to prove dishonest behaviour using the operator's own signed messages.
This design offers flexibility in balancing various trade-offs, including transaction costs versus rounds of interaction, prover versus verifier computational costs, and precomputation requirements versus protocol rounds. While BitVMX can still utilize signature equivocations like BitVM, they are not fundamental to its operation, resulting in a more streamlined protocol.
The TOOP protocol requires the following two properties from the BitVMX challenge-response system, which we formalize as the ideal functionality $\mathcal{F}_{\text{BitVMX}}$ in Section 4:

(1) Computational soundness: If the claimed statement is false (i.e., no valid burn occurred, or the group selection is invalid) and at least one verifier is honest, the claim is rejected with overwhelming probability in κ.
(2) Completeness: If the claimed statement is true and the prover is honest, the claim is accepted with overwhelming probability.

These are the minimal black-box properties needed for TOOP's transfer authenticity, double-spend prevention, and liveness. Any challenge-response verification system satisfying these properties-whether BitVMX, BitVM2 [14], or a future alternative-can serve as the underlying engine.

2.8. **Cryptographic components.** Let us denote with $\mathcal{M}$ and $\mathcal{C}$ the sets of messages and ciphertexts, respectively. We assume that the following data is public: an elliptic curve E with a prime q such that the discrete logarithm problem is hard over $E(\mathbb{Z}_q)$, a hash function $H : \{0,1\}^* \to \{0,1\}^{l_1+l_2}$ (applied to bitstring encodings of curve points and auxiliary data), for parameters $l_1, l_2 > 0$, and a MAC function $\text{MAC} : \mathcal{C} \to \mathbb{Z}_q$. Let $Sym = (\varepsilon, \delta)$ be a symmetric encryption mechanism. This protocol involves a phase where operators set $1-1$ communications with owners using the ECIES protocol [8], which we introduce informally below.

Algorithm 1 ECIES.KeyGen

1: Given a generator $A \in E(\mathbb{Z}_q)$ of order q: sample $s \xleftarrow{\$} \mathbb{Z}_q$ and compute $B = s \cdot A$
2: **Output:** $\mathsf{pk} = (A, B)$, $\mathsf{sk} = s$

Algorithm 2 ECIES.Encrypt(m, pk)

1: Parse $\mathsf{pk} = (A, B)$, choose random $k \in [1, q-1]$
2: Compute $R = k \cdot A$, $Z = k \cdot B$, $\kappa_E \,\|\, \kappa_M = H(R, Z)$
3: Compute $C = \varepsilon_{\kappa_E}(m)$, $t = \text{MAC}_{\kappa_M}(C)$
4: **return** (R, C, t)

Algorithm 3 ECIES.Decrypt(C, sk)

1: Compute $Z = \mathsf{sk} \cdot R$, $\kappa_E \,\|\, \kappa_M = H(R, Z)$, $t' = \text{MAC}_{\kappa_M}(C)$
2: **if** $t \neq t'$ **then reject** C
3: **else return** $m = \delta_{\kappa_E}(C)$
4: **end if**

This protocol also makes use of MuSig2 [17] for multi-signatures involving the committee. In this setting we consider a point $G \in E(\mathbb{Z}_q)$ of prime order q, and two hash functions $H_{\text{agg}} : \{0,1\}^* \to \mathbb{Z}_q$, $H_{\text{non}} : \{0,1\}^* \to \mathbb{Z}_q$, and $H_{\text{sig}} : E(\mathbb{Z}_q) \times E(\mathbb{Z}_q) \times \mathcal{M} \to \mathbb{Z}_q$.

Algorithm 4 MuSig.KeyGen

1: Each participant O_i selects secret key $k_i \in \mathbb{Z}_q$ and computes $K_i = k_i \cdot G \in E(\mathbb{Z}_q)$
2: Each O_i computes $a_i = H_{\text{agg}}(K_i, \{K_1, \ldots, K_n\}) \in \mathbb{Z}_q$
3: Aggregate public key: $\mathsf{pk}_{\mathcal{O}} = K = \sum_{i=1}^{n} a_i \cdot K_i \in E(\mathbb{Z}_q)$
4: Implied aggregate secret key: $\mathsf{sk}_{\mathcal{O}} = k = \sum_{i=1}^{n} a_i \cdot k_i \mod q$

Algorithm 5 MuSig.Sign(m)

1: **Round 1:** Each O_i generates $r_{i1}, r_{i2} \in \mathbb{Z}_q$, computes $R_{i1} = r_{i1} \cdot G$, $R_{i2} = r_{i2} \cdot G$
2: Each O_i broadcasts (R_{i1}, R_{i2})
3: **Round 2:** All compute $b = H_{\text{non}}(R_{11} \,\|\, R_{12} \,\|\, \ldots \,\|\, R_{n1} \,\|\, R_{n2}, m) \in \mathbb{Z}_q$
4: Each O_i computes $R_i = R_{i1} + b \cdot R_{i2}$ and all compute $R = \sum_{i=1}^{n} R_i$
5: All compute challenge $c = H_{\text{sig}}(\mathsf{pk}_{\mathcal{O}}, R, m) \in \mathbb{Z}_q$
6: Each O_i computes and broadcasts $s_i = r_{i1} + b \cdot r_{i2} + c \cdot a_i \cdot k_i \mod q$
7: Final signature: $\sigma = (R, s) = (R, \sum_{i=1}^{n} s_i \mod q)$

Algorithm 6 MuSig.Verification(m, σ)

1: Compute $c = H_{\text{sig}}(\mathsf{pk}_{\mathcal{O}}, R, m)$
2: Check if $s \cdot G = R + c \cdot \mathsf{pk}_{\mathcal{O}}$
3: **if** equality holds **then return** valid
4: **else return** invalid
5: **end if**

With the cryptographic primitives established, we now describe the protocol.

3. Protocol overview

Let us assume that a user Θ_0 has an asset (UTXO) on Bitcoin and wants to wrap it to use it on a secondary blockchain with smart contracts support.
A group of $n \geq 1$ operators is in charge of managing the protocol and have published an aggregate address to receive assets and help with the wrapping, following MuSig2.

3.1. **Locking keys and power set.** For each locked asset in Bitcoin, the operators generate a corresponding list of private and public keys. To this end, each operator O_i generates a random secret key $k_i \in \mathbb{Z}_q$ and publishes the public key $K_i = k_i \cdot G$, for $G \in E(\mathbb{Z}_q)$ of prime order q. The MuSig2 aggregation coefficient for O_i is defined as $a_i := H_{\text{agg}}(K_i, \{K_1, \ldots, K_n\})$, computed once over the full operator set $\mathcal{O}$ and fixed for all subsets. The *key share* of O_i is $\mathsf{sh}_i := a_i \cdot k_i \bmod q$. For any non-empty subset $S \subseteq \mathcal{O}$, the aggregate secret key is $\mathsf{sk}_S = \sum_{j \in S} \mathsf{sh}_j$ with corresponding public key $\mathsf{pk}_S = \sum_{j \in S} a_j K_j = \mathsf{sk}_S \cdot G$.

Let ID be the power set of all the subsets of operators which will collaborate with Θ_1 in forthcoming steps of the protocol.
In forthcoming steps, users will have the ability to consider elements of ID, and define both their aggregate public key and the associated signing key. This can be done restricting the aggregation process to the involved operators, where the coefficients are computed following MuSig2 and using the public keys of the willing operators.

3.2. **Fundamental phases.** We can divide the protocol in three phases, locking, asset usage, and unlocking, which are described below.

(1) Locking: ensures the secure transition of assets from Bitcoin into its wrapped form on the second blockchain.
   (a) A committee of operators generates, and shares, an aggregate address on Bitcoin following MuSig2.
   (b) Θ_0 transfers the asset to the operator's aggregate address on Bitcoin using a *lock request* transaction, which includes:
      (i) The assets to be locked.
      (ii) Protocol fees to support Bitcoin and the second blockchain operations.
      (iii) The second blockchain address selected to receive the wrapped assets.

      The lock request transaction offers two potential outcomes:
      (i) It can be redeemed by all operators to complete the lock process.
      (ii) If not accepted within a specified time-frame, the owner can reclaim the asset after a time-lock expires.
   (c) The user communicates with the operators off-chain, sharing details of the lock request transaction.
   (d) The operators identify the request and initiate the setup process, which involves:
      (i) Preparing BitVMX presigned transactions and secrets.
      (ii) Creating aggregate keys for the transfer of ownership. This is done using MuSig2.
   (e) The operators create a BitVMX program that will validate an unlocking request in the future, and will transfer the asset into one of the predefined $2^n - 1$ addresses (corresponding to the non-empty subsets of $\mathcal{O}$).
   (f) A wrapped asset linking to Θ_0 original asset is created on the second blockchain.
(2) Asset usage: in this phase, Θ_0 uses the asset on the secondary chain, possibly transferring it to other users or smart contracts.
(3) Unlocking: Two clarifications are needed here. *Network model*: the unlocking phase assumes a synchronous network with known upper bound Δ on message delivery time. Specifically, any message sent by an honest party is delivered to all other parties within time Δ. This assumption ensures that Θ_1 receives key shares from all honest operators before proceeding with subset selection. *Timing rule*: upon announcing the intention to unlock, Θ_1 waits for time $2 \cdot \Delta$ before selecting a subset. This guarantees that shares from all honest operators have been received, assuming the network bound holds.

(a) Θ_1 initiates the transfer by sending the wrapped asset to the smart contract for burning. This is done by submitting $\text{Burn}(X, \widetilde{\mathsf{pk}}_1)$ to the bridge contract on BC_2, where $\widetilde{\mathsf{pk}}_1$ is a fresh ECIES public key generated by Θ_1 for encrypting the key share transfer.

(b) Operators monitoring the smart contract will detect the burn request and verify the asset origin. The operators will use the provided public key to encrypt their shares of the aggregated key for that UTXO. In particular: each operator O_i encrypts its key share $\mathsf{sh}_i = a_i \cdot k_i \bmod q$ under $\widetilde{\mathsf{pk}}_1$ using ECIES, producing $C_i = \text{ECIES.Encrypt}(\widetilde{\mathsf{pk}}_1, \mathsf{sh}_i)$. Since the aggregation coefficients $a_i = H_{\text{agg}}(K_i, \{K_1, \ldots, K_n\})$ are fixed over the full operator set $\mathcal{O}$ (see Section 3.1), sh_i is independent of the subset S. For any $S \ni O_i$, the aggregate secret key is $\mathsf{sk}_S = \sum_{j \in S} \mathsf{sh}_j$, with corresponding public key $\mathsf{pk}_S = \sum_{j \in S} a_j K_j$. A single ciphertext per operator suffices for all $2^n - 1$ non-empty subsets.

(c) Θ_1 initiates off-chain communication with the operators to retrieve their encrypted shares and associated public keys. These shares are encrypted using the public key provided by Θ_1. Here, the cryptographic scheme in use is ECIES.
Θ_1 verifies each decrypted share sh_i by checking $\mathsf{sh}_i \cdot G = a_i \cdot K_i$, where a_i and K_i are the public aggregation coefficient and public key of operator O_i established during setup. Shares failing this check are discarded. This verification detects corrupted operators who send invalid or manipulated shares, but it does *not* certify honesty: a corrupted operator may send a valid share while simultaneously leaking it to an adversary. The security guarantee is that the adversary cannot reconstruct sk_S for any subset S containing the honest operator, because the honest operator's share remains encrypted under $\widetilde{\mathsf{pk}}_1$ (see Section 4.8, Corollary 4.30).

(d) Θ_1 verifies that the decrypted keys match the public keys linked to the locked UTXO. This allows the identification of the subset of valid keys and assign an element in ID to this subset.

(e) Θ_1 sends a message to the smart-contract selecting a group-id which corresponds to the aggregate public key of the subset of operators who sent valid keys.

(f) An operator starts the execution of the BitVMX unlocking program on Bitcoin, giving as input the selected group-id as well as a zero-knowledge proof that the owner burned and selected that group-id on a smart-contract transaction.

(g) If step (f) is successful, the locked asset is sent to address(S).

(h) Θ_1 calculates an aggregated private key for address(S) and transfers the asset to one address of his choice.

3.3. **Interaction between owners and operators.** Once detected the burning transaction in the smart contract, a subset of operators set communication with Θ_1 using the ECIES scheme. This communication is possible because Θ_1 generates a key pair, specific to $1 - 1$ communications, given by a random secret key $\mathsf{sk}_c \in \mathbb{Z}_q$, and

a public key $(G, \mathsf{sk}_c \cdot G)$. After checking the validity of the received secrets, Θ_1 can derive a set S in ID and compute the associated aggregated key following MuSig2.

4. UC Security analysis

The security of the Transfer of Ownership Protocol is established through a Universally Composable (UC) analysis following the framework of Canetti [6]. We adopt the UC approach for three reasons. First, TOOP composes three independent cryptographic primitives (MuSig2 for multi-signatures, ECIES for public-key encryption, and BitVMX for optimistic verification) across two blockchain ledgers. The UC composition theorem provides a rigorous guarantee that the security of each primitive is preserved under composition, replacing the need for ad-hoc arguments about key-space separation and hash-domain independence. Second, the UC ideal functionality $\mathcal{F}_{\text{TOOP}}$ forces a precise specification of every protocol interface, every leakage channel, and every timing guarantee. In particular, it resolves the relationship between operator key shares and MuSig2 aggregation coefficients, which requires a formal treatment to ensure that a single encrypted share per operator suffices for all operator subsets. Third, the UC formulation cleanly separates the protocol's security requirements (captured by $\mathcal{F}_{\text{TOOP}}$) from the assumptions on sub-protocols (captured by $\mathcal{F}_{\text{MuSig}}$, $\mathcal{F}_{\text{ECIES}}$, $\mathcal{F}_{\text{BitVMX}}$, $\mathcal{F}_{\text{BC}_1}$, $\mathcal{F}_{\text{BC}_2}$). This modularity means that any future replacement of a sub-protocol (for instance,, substituting BitVM2 for BitVMX, or upgrading the MuSig variant) preserves the top-level security guarantee, provided the replacement UC-realizes the same ideal functionality. The six security properties from the previous version of this paper (asset validity, transfer authenticity, double-spend prevention, ownership transfer security, 1-out-of-n security, and liveness) are recovered as corollaries of the main UC theorem (Corollaries 4.27–4.33).

4.1. Key share definition and properties.

Before defining the ideal functionalities, we must resolve the key-share definition gap, indeed: each operator O_i sends a *single key share* (which we will denote sh_i) such that for any subset $S \ni O_i$, the aggregate secret key is $\mathsf{sk}_S = \sum_{j \in S} s_j$. Simultaneously, it invokes MuSig2 aggregation with coefficients $a_i = H_{\text{agg}}(K_i, \{K_1, \ldots, K_n\})$. These two descriptions are inconsistent under standard MuSig2, where the aggregation coefficients depend on the participating set. We resolve this as follows.

Definition 4.1 (Fixed-reference-set key shares)**.** Let $\mathcal{O} = \{O_1, \ldots, O_n\}$ be the full operator set. During setup, each operator O_i generates a random secret key $k_i \xleftarrow{\$} \mathbb{Z}_q$ and publishes $K_i = k_i \cdot G$. The MuSig2 aggregation coefficients with fixed reference set $\mathcal{O}$ are: $a_i := H_{\text{agg}}\big(K_i, \{K_1, \ldots, K_n\}\big) \in \mathbb{Z}_q, \forall\, i \in [n]$. The coefficients are computed once over the full set $\mathcal{O}$ and remain fixed for all subsets.
The *operator key share* of O_i is defined as: $\mathsf{sh}_i := a_i \cdot k_i \mod q$. For any non-empty subset $S \subseteq \mathcal{O}$, the *aggregate secret key for* S is $\mathsf{sk}_S := \sum_{j \in S} \mathsf{sh}_j = \sum_{j \in S} a_j \cdot k_j \mod q$, with corresponding *aggregate public key*: $\mathsf{pk}_S := \sum_{j \in S} a_j \cdot K_j = \mathsf{sk}_S \cdot G$.

Remark 4.2 (Deviation from standard MuSig2 subset instantiation)**.** In standard MuSig2, when a subset $S \subseteq \mathcal{O}$ wishes to produce a multi-signature, the aggregation coefficients are recomputed as $a_i^{(S)} = H_{\text{agg}}(K_i, \{K_j\}_{j \in S})$, which change with S. Our construction fixes the reference set to $\mathcal{O}$, making a_i independent of S.
This deviation has two consequences that require justification:

(1) **Universal shares**: Since $\mathsf{sh}_i = a_i \cdot k_i$ does not depend on S, a single encrypted share per operator suffices for all $2^n - 1$ non-empty subsets. This is the key property enabling the protocol's efficiency.
(2) **Security**: The aggregate public key $\mathsf{pk}_S = \sum_{j \in S} a_j K_j$ is *not* the standard MuSig2 aggregate key for the subset S. Although pk_S is not the standard MuSig2 aggregate key for the set S, the pair $(\mathsf{sk}_S, \mathsf{pk}_S)$ satisfies $\mathsf{pk}_S = \mathsf{sk}_S \cdot G$ and thus constitutes a valid Schnorr key pair. The Schnorr unforgeability reduction (Lemma 4.4) applies to this pair because the proof depends only on the DLP hardness in $E(\mathbb{Z}_q)$, not on the MuSig2 aggregation structure. At spending time, only Θ_1 (who holds all shares of S) produces the signature (no multi-party signing protocol is involved).

Lemma 4.3 (Subset key reconstruction security)**.** *Under the discrete logarithm assumption in $E(\mathbb{Z}_q)$, given $\{K_j\}_{j \in [n]}$ and $\{\mathsf{sh}_j\}_{j \in S'}$ for any $S' \subsetneq S$, no PPT adversary can compute sk_S with non-negligible probability.*

Proof. Suppose adversary $\mathcal{A}$ can compute $\mathsf{sk}_S = \sum_{j \in S} \mathsf{sh}_j$ given $\{\mathsf{sh}_j\}_{j \in S'}$ for $S' \subsetneq S$. Since $S' \subseteq S \setminus \{i^*\}$, let $i^* \in S \setminus S'$ be any index whose share is unknown to $\mathcal{A}$; existence is guaranteed by $S' \subsetneq S$. (In the TOOP setting, i^* corresponds to an honest operator in S, guaranteed by $|C| \leq n - 1$.) Providing $\mathcal{A}$ with the additional shares $\{\mathsf{sh}_j\}_{j \in (S \setminus S') \setminus \{i^*\}}$ can only increase its success probability (more information cannot decrease a PPT algorithm's advantage). We may therefore assume without loss of generality that $\mathcal{A}$ knows $\{\mathsf{sh}_j\}_{j \in S \setminus \{i^*\}}$. Then $\mathcal{A}$ can compute $\mathsf{sh}_{i^*} = \mathsf{sk}_S - \sum_{j \in S \setminus \{i^*\}} \mathsf{sh}_j$. Since $\mathsf{sh}_{i^*} = a_{i^*} \cdot k_{i^*}$ and a_{i^*} is publicly computable, $\mathcal{A}$ obtains $k_{i^*} = a_{i^*}^{-1} \cdot \mathsf{sh}_{i^*} \mod q$ (noting $a_{i^*} \neq 0$ with overwhelming probability under the random oracle model for H_{agg}). This yields k_{i^*} such that $K_{i^*} = k_{i^*} \cdot G$, solving a discrete logarithm instance. □

Lemma 4.4 (Schnorr unforgeability under subset keys)**.** *Under the discrete logarithm assumption and the random oracle model, Schnorr signatures under public key $\mathsf{pk}_S = \sum_{j \in S} a_j K_j$ are existentially unforgeable under chosen-message attacks when the signing key $\mathsf{sk}_S = \sum_{j \in S} a_j k_j$ is held by a single party.*

Proof. This is standard Schnorr signature security. The public key pk_S is a group element in $E(\mathbb{Z}_q)$, and the signing key satisfies $\mathsf{pk}_S = \mathsf{sk}_S \cdot G$. The security reduction to the discrete logarithm problem proceeds identically to Pointcheval–Stern [20]: given a DLP challenge (G, Y), embed $Y = \mathsf{pk}_S$ and use the forking lemma to extract sk_S from two valid forgeries (R, s_1) and (R, s_2) with distinct challenges c_1, c_2, yielding $\mathsf{sk}_S = (s_1 - s_2)(c_2 - c_1)^{-1} \mod q$. □

4.2. **Security assumptions.** The security of this proposed protocol relies on several cryptographic assumptions and the robustness of the underlying primitives, in particular MuSig2 and ECIES. Below, there is an outline of the key assumptions and their implications for the protocol's security.
The protocol operates under the 1-out-of-n honesty assumption, which assumes that among n operators, at least one is honest and follows the protocol specification correctly. This assumption is critical for ensuring the integrity and correctness of the protocol, as the honest participant serves as a safeguard against malicious behaviour by the remaining $n - 1$ operators. The honest participant ensures that the protocol's

outputs are verified correctly, that BitVMX challenges are set when required, and that no adversarial manipulation goes undetected.
The protocol employs MuSig2 for the multisignatures involving the committee of operators, and ECIES for setting secure communications between owners and operators. It therefore inherits the following security assumptions:

(1) Discrete logarithm problem over elliptic curves (DLP): the hardness of computing discrete logarithms in the underlying elliptic curve group ensures that an adversary cannot forge signatures without knowledge of the private keys.
(2) Random oracle model (ROM): we assume that cryptographic hash functions behave as random oracles. This model is essential for proving the security of the scheme against chosen-message attacks.
(3) Elliptic curve Diffie-Hellman problem (ECDHP): the hardness of solving the Diffie-Hellman problem ensures that an adversary cannot obtain the shared secret used for symmetric key derivation.
(4) Indistinguishability under chosen-plaintext attack (IND-CPA): the symmetric encryption component of ECIES is assumed to be IND-CPA secure, ensuring that ciphertexts do not leak information about the plaintext.
(5) Existential unforgeability under chosen-message attacks (EUF-CMA): The MAC component in ECIES ensures that ciphertexts cannot be forged or tampered with without detection.

Remark 4.5. The UC composition theorem requires that each sub-protocol UC-realizes its corresponding ideal functionality. For $\mathcal{F}_{\text{BitVMX}}$ and the ledger functionalities $\mathcal{F}_{\text{BC}_1}$, $\mathcal{F}_{\text{BC}_2}$, we state this as an explicit assumption. For $\mathcal{F}_{\text{MuSig}}$ and $\mathcal{F}_{\text{ECIES}}$:

(1) Baum et al. [4] introduce the first UC ideal functionality for interactive multi-signatures and prove that game-based EUF-CMA security under concurrent sessions is equivalent to UC realizability. MuSig2 is proven EUF-CMA secure under concurrent sessions in the ROM [17]. MuSig2 UC-realizes the interactive multi-signature functionality. The *compute key share* interface of $\mathcal{F}_{\text{MuSig}}$ is a deterministic local computation: O_i computes $\mathsf{sh}_i = a_i \cdot k_i \bmod q$ from its own k_i and the public coefficient a_i. Since no interaction or communication occurs, and the output is derived entirely from O_i's local state, any protocol that UC-realizes the Baum et al. interactive multi-signature functionality also realizes $\mathcal{F}_{\text{MuSig}}$ as defined here. The additional interface adds no new simulator obligations: the simulator can compute $\mathsf{sh}_i = a_i k_i$ for corrupted operators and need not produce anything for honest operators (the share is returned only to O_i, not leaked to $\mathcal{S}$).
(2) ECIES is proven IND-CPA secure under ECDHP. Our $\mathcal{F}_{\text{ECIES}}$ follows the standard structure of Canetti's $\mathcal{F}_{\mathsf{PKE}}$ for IND-CPA, with the modification that decryption of adversarial ciphertexts for honest recipients uses real ECIES decryption. The UC realization argument is standard: the simulator encrypts dummy values for honest recipients (exactly as in our Hybrid H_2), and indistinguishability follows from IND-CPA security.

We emphasize that Theorem 4.22 is stated in the hybrid world, so it holds regardless of how the sub-protocols are realized. The composition step is where these realization results are consumed.

We observe that MuSig2 incorporates a key aggregation technique that mitigates rogue-key attacks, ensuring that malicious participants cannot manipulate their public keys to gain an unfair advantage during signature generation.
These assumptions are the conditions under which the sub-protocol ideal functionalities $\mathcal{F}_{\text{MuSig}}$, $\mathcal{F}_{\text{ECIES}}$, $\mathcal{F}_{\text{BitVMX}}$ are realized by their respective real-world protocols.

4.3. **Sub-protocol ideal functionalities.** The protocol Π_{TOOP} operates in the hybrid world with access to five ideal functionalities. We define each below.

4.3.1. *Ledger functionality $\mathcal{F}_{\text{BC}_1}$ (Bitcoin).* We model the Bitcoin ledger as an ideal functionality following the approach of Badertscher, Maurer, Tschudi, and Zikas [3]. We simplify their GUC treatment to the essential interfaces needed by TOOP.

Functionality 4.6 ($\mathcal{F}_{\text{BC}_1}$: Bitcoin Ledger)**.** $\mathcal{F}_{\text{BC}_1}$ is parameterized by a confirmation depth k_1, a block interval τ_1, and maintains an ordered set of confirmed transactions Ledger_1.

State:

- $\mathsf{UTXOSet}$: set of unspent transaction outputs, each of them with the form $(\mathsf{txid}, \mathsf{idx}, \mathsf{val}, \mathsf{script})$.
- Ledger_1: ordered list of confirmed transactions.
- $\mathsf{Mempool}$: set of pending transactions.

Interfaces:

(1) **Submit transaction** $(\mathsf{submit\text{-}tx}, \mathsf{sid}, \mathsf{tx})$ from party P:
  - Validate tx against $\mathsf{UTXOSet}$ (inputs exist, signatures valid, values balance).
  - If valid, add tx to $\mathsf{Mempool}$.
  - Send $(\mathsf{tx\text{-}submitted}, \mathsf{sid}, \mathsf{tx})$ to $\mathcal{S}$.

(2) **Read ledger** $(\mathsf{read}, \mathsf{sid})$ from party P:
  - Return $(\mathsf{ledger}, \mathsf{sid}, \mathsf{Ledger}_1)$ to P.

(3) **Advance** $(\mathsf{advance}, \mathsf{sid})$ from $\mathcal{S}$ (modeling block production):
  - Select transactions from $\mathsf{Mempool}$ (adversary controls ordering but cannot prevent inclusion within time bound Δ_1).
  - Append selected transactions to Ledger_1.
  - Update $\mathsf{UTXOSet}$ accordingly.
  - For each included transaction tx, send $(\mathsf{tx\text{-}confirmed}, \mathsf{sid}, \mathsf{tx})$ to all parties.

(4) **Confirmation query** $(\mathsf{confirm?}, \mathsf{sid}, \mathsf{txid})$ from party P:
  - Return $(\mathsf{confirmed}, \mathsf{sid}, \mathsf{txid}, d)$ where d is the confirmation depth of txid in Ledger_1 (or $\perp$ if not found).

Timing guarantee: Any valid transaction submitted by an honest party is included in Ledger_1 within time $\Delta_1 = k_1 \cdot \tau_1$.

4.3.2. *Ledger functionality $\mathcal{F}_{\text{BC}_2}$ (Secondary chain).*

Functionality 4.7 (Secondary-Chain Ledger with Smart Contracts)**.** $\mathcal{F}_{\text{BC}_2}$ extends a standard ledger functionality with smart contract execution. It is parameterized by confirmation depth k_2 and block interval τ_2.

State (extends $\mathcal{F}_{\mathrm{BC}_1}$ with):

- $\mathsf{ContractState}[\mathsf{cid}]$: state of each deployed smart contract.
- $\mathsf{WrappedAssets}[\mathsf{cid}]$: mapping from asset identifiers to ownership records.
- $\mathsf{BurnLog}[\mathsf{cid}]$: set of burned asset identifiers with associated public keys.

Additional interfaces (beyond standard ledger):

(1) **Mint wrapped asset** $(\mathsf{mint}, \mathsf{sid}, \mathsf{cid}, \mathsf{assetId}, \mathsf{owner})$ from party P:
- Execute the bridge contract logic at cid. If the mint preconditions are met (valid lock proof), set $\mathsf{WrappedAssets}[\mathsf{cid}][\mathsf{assetId}] \leftarrow \mathsf{owner}$.
- Send $(\mathsf{minted}, \mathsf{sid}, \mathsf{assetId}, \mathsf{owner})$ to $\mathcal{S}$.

(2) **Burn wrapped asset** $(\mathsf{burn}, \mathsf{sid}, \mathsf{cid}, \mathsf{assetId}, \widetilde{\mathsf{pk}}_1)$ from party Θ_1:
- Verify $\mathsf{WrappedAssets}[\mathsf{cid}][\mathsf{assetId}] = \Theta_1$.
- Verify $\mathsf{assetId} \notin \mathsf{BurnLog}[\mathsf{cid}]$.
- If valid, then delete $\mathsf{WrappedAssets}[\mathsf{cid}][\mathsf{assetId}]$ and add $(\mathsf{assetId}, \widetilde{\mathsf{pk}}_1)$ to $\mathsf{BurnLog}[\mathsf{cid}]$.
- Send $(\mathsf{burned}, \mathsf{sid}, \mathsf{assetId}, \widetilde{\mathsf{pk}}_1)$ to all parties and $\mathcal{S}$.

(3) **Group selection** $(\mathsf{select\text{-}group}, \mathsf{sid}, \mathsf{cid}, \mathsf{assetId}, \mathsf{gid})$ from party Θ_1:
- Verify $(\mathsf{assetId}, \cdot) \in \mathsf{BurnLog}[\mathsf{cid}]$.
- Record $\mathsf{GroupSelection}[\mathsf{assetId}] \leftarrow \mathsf{gid}$.
- Send $(\mathsf{group\text{-}selected}, \mathsf{sid}, \mathsf{assetId}, \mathsf{gid})$ to all parties and $\mathcal{S}$.

Timing guarantee: Analogous to $\mathcal{F}_{\mathrm{BC}_1}$ with parameters (k_2, τ_2, Δ_2).

4.3.3. *Multi-signature functionality* $\mathcal{F}_{\mathrm{MuSig}}$.

Functionality 4.8 ($\mathcal{F}_{\mathrm{MuSig}}$: MuSig2 Multi-Signature)**.** $\mathcal{F}_{\mathrm{MuSig}}$ is parameterized by a group $(E(\mathbb{Z}_q), G, q)$ and hash functions H_{agg}, H_{sig}.

State:

- Keys: mapping from party identities to (k_i, K_i) pairs.
- $\mathsf{AggCoeff}$: mapping from party identities to aggregation coefficients a_i.

Interfaces:

(1) **Key generation** $(\mathsf{keygen}, \mathsf{sid})$ from operator O_i:
- If O_i is honest: sample $k_i \xleftarrow{\$} \mathbb{Z}_q$, set $K_i = k_i \cdot G$.
- If O_i is corrupted: receive (k_i, K_i) from $\mathcal{S}$ (rogue-key protection: verify $K_i = k_i \cdot G$).
- Store $\mathsf{Keys}[O_i] \leftarrow (k_i, K_i)$.
- Send $(\mathsf{pubkey}, \mathsf{sid}, O_i, K_i)$ to all parties and $\mathcal{S}$.
- Once all n operators have registered: $a_i = H_{\mathrm{agg}}(K_i, \{K_1, \ldots, K_n\})$ for all i, store in $\mathsf{AggCoeff}$.
- Output $(\mathsf{setup\text{-}complete}, \mathsf{sid}, \{(O_i, K_i)\}_{i\in[n]}, \{a_i\}_{i\in[n]})$ to all parties.

(2) **Compute key share** $(\mathsf{compute\text{-}share}, \mathsf{sid})$ from operator O_i:
- Compute $\mathsf{sh}_i = a_i \cdot k_i \mod q$.
- Return $(\mathsf{share}, \mathsf{sid}, O_i, \mathsf{sh}_i)$ to O_i.
- Send $(\mathsf{share\text{-}computed}, \mathsf{sid}, O_i)$ to $\mathcal{S}$ (leaks identity, not value).

(3) **Compute subset aggregate public key** $(\mathsf{agg\text{-}pk}, \mathsf{sid}, S)$ from any party P:
- Compute $\mathsf{pk}_S = \sum_{j\in S} a_j \cdot K_j$.

- Return $(\mathsf{agg\text{-}pk\text{-}result}, \mathsf{sid}, S, \mathsf{pk}_S)$ to P.

Remark 4.9 (Rogue-key protection)**.** The functionality requires corrupted operators to provide valid key pairs (k_i, K_i) with $K_i = k_i \cdot G$. This models MuSig2's built-in rogue-key mitigation through the H_{agg} coefficient computation. In practice, this is enforced by requiring a proof of knowledge of k_i (for instance,, a Schnorr proof) during key registration.

4.3.4. *Public-key encryption functionality* $\mathcal{F}_{\text{ECIES}}$.

Functionality 4.10 ($\mathcal{F}_{\text{ECIES}}$: ECIES Public-Key Encryption)**.** $\mathcal{F}_{\text{ECIES}}$ provides IND-CPA secure public-key encryption.

State:

- $\mathsf{KeyPairs}$: mapping from identities to $(\mathsf{sk}, \mathsf{pk})$ pairs.
- $\mathsf{DecTable}$: mapping from ciphertext handles to plaintexts (for honest recipients).

Interfaces:

(1) **Key generation** $(\mathsf{enc\text{-}keygen}, \mathsf{sid})$ from party P:
- If P is honest: sample $\mathsf{sk}_P \xleftarrow{\$} \mathbb{Z}_q$, compute $\mathsf{pk}_P = \mathsf{sk}_P \cdot G$.
- If P is corrupted: receive $(\mathsf{sk}_P, \mathsf{pk}_P)$ from $\mathcal{S}$.
- Store $\mathsf{KeyPairs}[P] \leftarrow (\mathsf{sk}_P, \mathsf{pk}_P)$.
- Output $(\mathsf{enc\text{-}pubkey}, \mathsf{sid}, P, \mathsf{pk}_P)$ to all parties and $\mathcal{S}$.

(2) **Encrypt** $(\mathsf{encrypt}, \mathsf{sid}, \mathsf{pk}_P, m)$ from party Q:
- If P is honest: compute $c \leftarrow \mathsf{ECIES.Encrypt}(\mathsf{pk}_P, m)$. Store $\mathsf{DecTable}[c] \leftarrow m$. Return $(\mathsf{ciphertext}, \mathsf{sid}, c)$ to Q. Send $(\mathsf{encrypted}, \mathsf{sid}, |m|)$ to $\mathcal{S}$ (leaks only length).
- If P is corrupted, then compute $c \leftarrow \mathsf{ECIES.Encrypt}(\mathsf{pk}_P, m)$. Return $(\mathsf{ciphertext}, \mathsf{sid}, c)$ to Q. Send $(\mathsf{encrypted}, \mathsf{sid}, c, m)$ to $\mathcal{S}$ (the adversary holds sk_P and can run real ECIES decryption, so full leakage is unavoidable and matches the real-world view).

(3) **Decrypt** $(\mathsf{decrypt}, \mathsf{sid}, c)$ from party P:
- If P is honest and $c \in \mathsf{DecTable}$: return $(\mathsf{plaintext}, \mathsf{sid}, \mathsf{DecTable}[c])$ to P.
- If P is honest and $c \notin \mathsf{DecTable}$: compute $m \leftarrow \mathsf{ECIES.Decrypt}(\mathsf{sk}_P, c)$. Return $(\mathsf{plaintext}, \mathsf{sid}, m)$ to P.
- If P is corrupted: forward to $\mathcal{S}$.

Remark 4.11. The leakage to $\mathcal{S}$ when the recipient is honest consists only of the message length $|m|$. This models the IND-CPA security of ECIES: the simulator can produce ciphertexts indistinguishable from real ones without knowing the plaintext.

4.3.5. *Verification functionality* $\mathcal{F}_{\text{BitVMX}}$. This functionality specifies the black-box properties that the TOOP protocol requires from the BitVMX challenge-response system.

Functionality 4.12 ($\mathcal{F}_{\text{BitVMX}}$: Optimistic Verification)**.** $\mathcal{F}_{\text{BitVMX}}$ is parameterized by a verification predicate $V : \{0,1\}^* \times \{0,1\}^* \to \{0,1\}$, a challenge window Δ_{chal}, and a set of verifiers $\mathcal{V} \subseteq \mathcal{O}$.

State:

- Claims: mapping from claim identifiers to (stmt, witness, status) triples.
- Outcomes: mapping from claim identifiers to {accepted, rejected}.

Interfaces:

(1) **Submit claim** (claim, sid, claimId, stmt, w) from prover P:
- Store Claims[claimId] $\leftarrow$ (stmt, w, pending).
- Send (claim-submitted, sid, claimId, stmt) to all parties and $\mathcal{S}$.
- Start challenge timer $T_{\mathsf{claimId}} \leftarrow \Delta_{\mathsf{chal}}$.

(2) **Challenge** (challenge, sid, claimId) from verifier $O_j \in \mathcal{V}$:
- Requires $T_{\mathsf{claimId}} > 0$ (within challenge window).
- Retrieve (stmt, $w, \cdot$) $\leftarrow$ Claims[claimId].
- Compute $b \leftarrow V(\mathsf{stmt}, w)$.
- If $b = 0$ (invalid claim): set Outcomes[claimId] $\leftarrow$ rejected. The prover forfeits its bond.
- Send (challenge-result, sid, claimId, b) to all parties and $\mathcal{S}$.

(3) **Finalize** (finalize, sid, claimId) (automatic when T_{claimId} expires):
- If no successful challenge occurred: set Outcomes[claimId] $\leftarrow$ accepted.
- Send (finalized, sid, claimId, Outcomes[claimId]) to all parties and $\mathcal{S}$.

Deterministic soundness: If $V(\mathsf{stmt}, w) = 0$ and at least one verifier $O_j \in \mathcal{V}$ is honest, then Outcomes[claimId] $\leftarrow$ rejected (deterministic). If $V(\mathsf{stmt}, w) = 0$ and *all* verifiers are corrupted (no honest challenge arrives before T_{claimId} expires), $\mathcal{S}$ may set Outcomes[claimId] $\leftarrow$ accepted.

Deterministic completeness: If $V(\mathsf{stmt}, w) = 1$ and the prover is honest, then Outcomes[claimId] $\leftarrow$ accepted (deterministic).

Remark 4.13 (Realization of $\mathcal{F}_{\mathrm{BitVMX}}$)**.** The deterministic guarantees of $\mathcal{F}_{\mathrm{BitVMX}}$ are the *ideal-world* specification. The statement that the real BitVMX system UC-realizes $\mathcal{F}_{\mathrm{BitVMX}}$ is where the "with overwhelming probability" qualifier appears: any violation of soundness or completeness by the real protocol would let $\mathcal{Z}$ distinguish real from ideal.

Remark 4.14 (BitVMX as a black box)**.** $\mathcal{F}_{\mathrm{BitVMX}}$ makes explicit the requirements that the TOOP protocol places on the BitVMX system: computational soundness (false claims are rejected when an honest verifier exists) and completeness (true claims are accepted). These are the minimal properties needed for TOOP's transfer authenticity and liveness guarantees. Any system satisfying this interface-whether BitVMX, BitVM2, or a future alternative-can be used.

4.4. **The TOOP Ideal Functionality.** We now define the ideal functionality $\mathcal{F}_{\mathrm{TOOP}}$ that captures all security guarantees of the protocol in a single object.

Functionality 4.15 ($\mathcal{F}_{\mathrm{TOOP}}$: Transfer of Ownership Protocol)**.** $\mathcal{F}_{\mathrm{TOOP}}$ is parameterized by operator set $\mathcal{O} = \{O_1, \ldots, O_n\}$, security parameter κ, corruption bound $n-1$, and time bounds $\Delta_{\mathsf{lock}}, \Delta_{\mathsf{mint}}, \Delta_{\mathsf{burn}}, \Delta_{\mathsf{share}}, \Delta_{\mathsf{unlock}}$.

Initialization. Upon first activation, $\mathcal{F}_{\text{TOOP}}$ internally generates key material: for each $O_i \in O$, sample $k_i \xleftarrow{\$} \mathbb{Z}_q$, compute $K_i = k_i \cdot G$, $a_i = H_{\text{agg}}(K_i, \{K_1, \ldots, K_n\})$, and $\mathsf{sh}_i = a_i k_i \bmod q$. For all non-empty $S \subseteq O$, compute $\mathsf{pk}_S = \sum_{j \in S} a_j K_j$. Store all values in internal state. Send $(\mathsf{setup\text{-}complete}, \mathsf{sid}, \{(O_i, K_i)\}, \{\mathsf{pk}_S\})$ to all parties and $\mathcal{S}$.

State:

- LockedAssets: mapping from ssid to $(\mathsf{utxo}, \Theta_0, \mathsf{status})$.
- WrappedAssets: mapping from ssid to $(\mathsf{assetId}, \mathsf{currentOwner})$.
- BurnRecords: mapping from ssid to $(\Theta_1, \widetilde{\mathsf{pk}}_1, \mathsf{status})$.
- ShareRecords: mapping from (ssid, O_i) to $\{\mathsf{delivered}, \mathsf{pending}\}$.
- GroupSelected: mapping from ssid to (gid, S).
- Completed: set of ssid values for completed transfers.

Interfaces:

(1) **Lock** $(\mathsf{Lock}, \mathsf{sid}, \mathsf{ssid}, \mathsf{utxo}, \Theta_0, \mathsf{addr}_{\mathsf{BC}_2})$ from Θ_0:
- **Asset validity check:** Verify that utxo is owned by Θ_0. Formally: if $\mathsf{utxo} \notin \mathsf{Assets}(\Theta_0, \mathsf{BC}_1)$, ignore the request.
- If valid: store $\mathsf{LockedAssets}[\mathsf{ssid}] \leftarrow (\mathsf{utxo}, \Theta_0, \mathsf{locked})$.
- Send $(\mathsf{lock\text{-}notification}, \mathsf{sid}, \mathsf{ssid}, \mathsf{utxo}, \mathsf{addr}_{\mathsf{BC}_2})$ to $\mathcal{S}$ (leaks: asset identity, destination chain address).
- $\mathcal{S}$ may delay up to Δ_{lock} before the lock is recorded.
- Output $(\mathsf{locked}, \mathsf{sid}, \mathsf{ssid})$ to Θ_0.

(2) **Mint** $(\mathsf{Mint}, \mathsf{sid}, \mathsf{ssid})$:
- **Transfer authenticity:** Requires $\mathsf{LockedAssets}[\mathsf{ssid}].\mathsf{status} = \mathsf{locked}$.
- If valid: store $\mathsf{WrappedAssets}[\mathsf{ssid}] \leftarrow (\mathsf{ssid}, \Theta_0)$ where we define $\Theta_0 = \mathsf{LockedAssets}[\mathsf{ssid}].\Theta_0$.
- Send $(\mathsf{mint\text{-}notification}, \mathsf{sid}, \mathsf{ssid})$ to $\mathcal{S}$.
- $\mathcal{S}$ may delay up to Δ_{mint}.
- Output $(\mathsf{minted}, \mathsf{sid}, \mathsf{ssid})$ to Θ_0.

(3) **Transfer on** BC_2 $(\mathsf{transfer}_{\mathsf{BC}_2}, \mathsf{sid}, \mathsf{ssid}, \Theta_{\mathsf{from}}, \Theta_{\mathsf{to}})$:
- Verify $\mathsf{WrappedAssets}[\mathsf{ssid}].\mathsf{currentOwner} = \Theta_{\mathsf{from}}$.
- Update $\mathsf{WrappedAssets}[\mathsf{ssid}].\mathsf{currentOwner} \leftarrow \Theta_{\mathsf{to}}$.
- Send $(\mathsf{transfer\text{-}notification}, \mathsf{sid}, \mathsf{ssid}, \Theta_{\mathsf{to}})$ to $\mathcal{S}$.

(4) **Burn** $(\mathsf{Burn}, \mathsf{sid}, \mathsf{ssid}, \Theta_1, \widetilde{\mathsf{pk}}_1)$ from Θ_1:
- **Ownership check:** Verify $\mathsf{WrappedAssets}[\mathsf{ssid}].\mathsf{currentOwner} = \Theta_1$.
- **Double-spend check:** Verify $\mathsf{ssid} \notin \mathsf{BurnRecords}$.
- If valid: store $\mathsf{BurnRecords}[\mathsf{ssid}] \leftarrow (\Theta_1, \widetilde{\mathsf{pk}}_1, \mathsf{active})$.
- Delete $\mathsf{WrappedAssets}[\mathsf{ssid}]$.
- Send $(\mathsf{burn\text{-}notification}, \mathsf{sid}, \mathsf{ssid}, \widetilde{\mathsf{pk}}_1)$ to all operators and $\mathcal{S}$.
- Output $(\mathsf{burned}, \mathsf{sid}, \mathsf{ssid})$ to Θ_1.

(5) **Distribute shares** $(\mathsf{ShareDist}, \mathsf{sid}, \mathsf{ssid}, O_i)$ from operator O_i:
- Requires $\mathsf{BurnRecords}[\mathsf{ssid}].\mathsf{status} = \mathsf{active}$.
- Let $\widetilde{\mathsf{pk}}_1 = \mathsf{BurnRecords}[\mathsf{ssid}].\widetilde{\mathsf{pk}}_1$.
- **If O_i is honest:** Retrieve sh_i (operator O_i's key share). Encrypt: $c_i \leftarrow \mathsf{ECIES.Encrypt}(\widetilde{\mathsf{pk}}_1, \mathsf{sh}_i)$. Record $\mathsf{ShareRecords}[(\mathsf{ssid}, O_i)] \leftarrow \mathsf{delivered}$.

Send $(\mathsf{share\text{-}delivered}, \mathsf{sid}, \mathsf{ssid}, O_i, c_i)$ to Θ_1.
Send $(\mathsf{share\text{-}delivered}, \mathsf{sid}, \mathsf{ssid}, O_i)$ to $\mathcal{S}$ (leaks identity, not content).

- **If O_i is corrupted:** $\mathcal{S}$ provides (c_i, sh_i). Record delivery. Send c_i to Θ_1.

Key share confidentiality: For honest O_i and honest Θ_1, the simulator $\mathcal{S}$ receives no information about sh_i beyond the fact that it was delivered.

(6) **Group selection** $(\mathsf{GroupSel}, \mathsf{sid}, \mathsf{ssid}, S)$ from Θ_1:
- Requires $\mathsf{ShareRecords}[(\mathsf{ssid}, O_j)] = \mathsf{delivered}$ for all $O_j \in S$. The real-world protocol has Θ_1 verify each share via $\mathsf{sh}_j \cdot G \stackrel{?}{=} a_j \cdot K_j$ (Protocol 4.19, step 5). In the ideal world, $\mathcal{F}_{\mathrm{TOOP}}$ accepts $\mathcal{S}$'s reported shares for corrupted operators without re-checking, since the simulator is responsible for maintaining consistency.
- Compute $\mathsf{gid} = \mathsf{SubsetToId}(S)$.
- Store $\mathsf{GroupSelected}[\mathsf{ssid}] \leftarrow (\mathsf{gid}, S)$.
- Send $(\mathsf{group\text{-}selected}, \mathsf{sid}, \mathsf{ssid}, \mathsf{gid})$ to all parties and $\mathcal{S}$.

(7) **Unlock** $(\mathsf{Unlock}, \mathsf{sid}, \mathsf{ssid}, \mathsf{addr}_{\Theta_1})$ from Θ_1:
- Requires $\mathsf{GroupSelected}[\mathsf{ssid}] = (\mathsf{gid}, S)$ for some S.
- Requires $\mathsf{BurnRecords}[\mathsf{ssid}] \neq \bot$.
- **Ownership transfer:** Transfer the locked UTXO to addr_{Θ_1} (an address chosen by Θ_1).
- Add ssid to Completed.
- Send $(\mathsf{unlocked}, \mathsf{sid}, \mathsf{ssid}, \mathsf{addr}_{\Theta_1})$ to Θ_1 and $\mathcal{S}$.

(8) **Abort / Reclaim** $(\mathsf{reclaim}, \mathsf{sid}, \mathsf{ssid})$ from Θ_0:
- Requires $\mathsf{LockedAssets}[\mathsf{ssid}].\mathsf{status} = \mathsf{locked}$ and time-lock has expired without minting.
- Return utxo to Θ_0.
- Set $\mathsf{LockedAssets}[\mathsf{ssid}].\mathsf{status} \leftarrow \mathsf{reclaimed}$.

Global invariants enforced by $\mathcal{F}_{\mathrm{TOOP}}$:

I1 **Asset validity:** Lock is accepted only if $\mathsf{utxo} \in \mathsf{Assets}(\Theta_0, \mathsf{BC}_1)$.

I2 **Transfer authenticity:** Mint requires a prior Lock; Unlock requires a prior Burn.

I3 **Double-spend prevention:** Each ssid can be burned at most once; each UTXO can be unlocked at most once (via UTXO consumption on BC_1).

I4 **Ownership transfer security:** Unlock delivers the asset to $\mathsf{Addr}(\Theta_1)$, an address chosen by Θ_1. In the real-world protocol, Θ_1 achieves this by first reconstructing sk_S from the received shares to spend from the intermediate address $\mathsf{Addr}(\mathsf{pk}_S)$. The ideal functionality abstracts this: only Θ_1 can trigger Unlock, and the UTXO is delivered to Θ_1's chosen address.

I5 **Key share confidentiality:** Honest operators' shares are never revealed to $\mathcal{S}$ when Θ_1 is honest.

I6 **Liveness:** If Θ_1 is honest and at least one operator is honest, the protocol completes within time $\Delta_{\mathsf{total}} = \Delta_{\mathsf{lock}} + \Delta_{\mathsf{mint}} + \Delta_{\mathsf{burn}} + \Delta_{\mathsf{share}} + \Delta_{\mathsf{unlock}}$.

4.5. **Protocol Π_{TOOP} in the Hybrid World.** We specify the protocol Π_{TOOP} in the $(\mathcal{F}_{\mathrm{MuSig}}, \mathcal{F}_{\mathrm{ECIES}}, \mathcal{F}_{\mathrm{BitVMX}}, \mathcal{F}_{\mathrm{BC}_1}, \mathcal{F}_{\mathrm{BC}_2})$-hybrid world. Each party's behavior is defined by the messages it sends to and receives from the ideal functionalities.

4.5.1. *Setup phase.*

Protocol 4.16 (Π_{TOOP}: Setup)**.** On input $(\mathsf{setup}, \mathsf{sid}, \mathcal{O})$, executed once per locked UTXO identified by ssid:

(1) Each operator $O_i \in \mathcal{O}$ sends $(\mathsf{keygen}, \mathsf{sid})$ to $\mathcal{F}_{\text{MuSig}}$.
(2) Upon receiving $(\mathsf{setup\text{-}complete}, \mathsf{sid}, \{(O_i, K_i)\}, \{a_i\})$ from $\mathcal{F}_{\text{MuSig}}$:
   - Each O_i sends $(\mathsf{compute\text{-}share}, \mathsf{sid})$ to $\mathcal{F}_{\text{MuSig}}$ to obtain $\mathsf{sh}_i = a_i \cdot k_i \mod q$.
   - O_i stores sh_i locally.
(3) Operators compute pk_S for every non-empty $S \subseteq \mathcal{O}$ by sending $(\mathsf{agg\text{-}pk}, \mathsf{sid}, S)$ to $\mathcal{F}_{\text{MuSig}}$ for each $S \in 2^{\mathcal{O}} \setminus \{\emptyset\}$.
(4) Operators compute the *full aggregate public key* $\mathsf{pk}_{\mathcal{O}} = \sum_{i=1}^{n} a_i K_i$ and derive the aggregate Bitcoin address $\mathsf{addr}_{\mathcal{O}} = \mathsf{PubKeyToAddr}(\mathsf{pk}_{\mathcal{O}})$.
(5) Operators prepare the BitVMX presigned transaction set: for each non-empty $S \subseteq \mathcal{O}$, they create a presigned transaction tx_S that spends the locked UTXO to $\mathsf{Addr}(\mathsf{pk}_S)$, conditional on $\mathcal{F}_{\text{BitVMX}}$ accepting the corresponding claim.

Remark 4.17. Steps (1) - (5) are executed independently for each locked UTXO. Distinct UTXOs have independent key families $\{(\mathsf{sk}_S, \mathsf{pk}_S)\}_{S \in \mathsf{ID}}$, ensuring that key shares learned during one unlock cannot be reused against a different UTXO.

4.5.2. *Lock phase.*

Protocol 4.18 (Π_{TOOP}: Lock)**.** On input $(\mathsf{Lock}, \mathsf{sid}, \mathsf{ssid}, \mathsf{utxo}, \Theta_0, \mathsf{addr}_{\mathsf{BC}_2})$:

(1) Θ_0 constructs a lock-request transaction $\mathsf{tx}_{\mathsf{lock}}$:
   - Input: utxo (owned by Θ_0).
   - Output 1: value of utxo sent to $\mathsf{addr}_{\mathcal{O}}$ (operator aggregate address).
   - Output 2: protocol fees.
   - Time-lock: Θ_0 can reclaim after timeout T_{lock} if operators do not proceed.
(2) Θ_0 sends $(\mathsf{submit\text{-}tx}, \mathsf{sid}, \mathsf{tx}_{\mathsf{lock}})$ to $\mathcal{F}_{\text{BC}_1}$.
(3) Θ_0 communicates $(\mathsf{lock\text{-}request}, \mathsf{ssid}, \mathsf{tx}_{\mathsf{lock}}, \mathsf{addr}_{\mathsf{BC}_2})$ to all operators (this is performed off-chain).
(4) Each honest operator O_i:
   - Reads $\mathcal{F}_{\text{BC}_1}$ to verify $\mathsf{tx}_{\mathsf{lock}}$ is confirmed with depth $\geq k_1$.
   - Verifies that the input UTXO is legitimately owned by Θ_0 (checks signatures and UTXO set).
   - If verification fails: O_i refuses to proceed, halting protocol progression.
(5) Upon successful verification by all participating operators: operators execute the setup process for ssid (presigned transactions, BitVMX program).
(6) Minting on BC_2: Any party (including Θ_0) submits a mint transaction to $\mathcal{F}_{BC_2}$ by providing a lock-inclusion proof.

4.5.3. *Asset usage phase.* During this phase, the wrapped asset exists on BC_2 and can be transferred among users via standard BC_2 transactions. The TOOP protocol is not involved. The current owner at any time is tracked by $\mathcal{F}_{\text{BC}_2}$.

4.5.4. *Unlock phase.*

Protocol 4.19 (Π_{TOOP}: Unlock)**.** On input $(\mathsf{Unlock}, \mathsf{sid}, \mathsf{ssid})$ from Θ_1 (the current owner of the wrapped asset on BC_2):

(1) Θ_1 generates an ECIES key pair by sending $(\mathsf{enc\text{-}keygen}, \mathsf{sid})$ to $\mathcal{F}_{\text{ECIES}}$, obtaining $(\widetilde{\mathsf{sk}}_1, \widetilde{\mathsf{pk}}_1)$.
(2) Θ_1 sends $(\mathsf{burn}, \mathsf{sid}, \mathsf{cid}, \mathsf{ssid}, \widetilde{\mathsf{pk}}_1)$ to $\mathcal{F}_{\text{BC}_2}$.
(3) Each operator O_i detects the burn event $(\mathsf{burned}, \mathsf{sid}, \mathsf{ssid}, \widetilde{\mathsf{pk}}_1)$ from $\mathcal{F}_{\text{BC}_2}$.
(4) **Share distribution:** Each operator O_i:
   - Verifies the burn transaction on $\mathcal{F}_{\text{BC}_2}$ (correct asset, correct contract).
   - Sends $(\mathsf{encrypt}, \mathsf{sid}, \widetilde{\mathsf{pk}}_1, \mathsf{sh}_i)$ to $\mathcal{F}_{\text{ECIES}}$, obtaining ciphertext c_i.
   - Sends (c_i, K_i) to Θ_1 via an off-chain, authenticated but not private channel. The adversary $\mathcal{A}$ may observe ciphertexts in transit. Privacy of key shares relies on the ECIES encryption, not on channel secrecy.
(5) **Share collection and verification:** Θ_1 waits for time 2Δ (synchrony bound), then for each received (c_i, K_i):
   - Sends $(\mathsf{decrypt}, \mathsf{sid}, c_i)$ to $\mathcal{F}_{\text{ECIES}}$, obtaining sh'_i.
   - Verifies: $\mathsf{sh}'_i \cdot G \stackrel{?}{=} a_i \cdot K_i$ (checks that the share is consistent with the public key and aggregation coefficient).
   - If verification succeeds: add O_i to the set of valid operators $\mathcal{V}$.
   - If verification fails: discard O_i's share.
(6) **Subset selection:** Θ_1 selects a subset $S \subseteq \mathcal{V}$ (any non-empty subset of valid operators).
(7) Θ_1 computes $\mathsf{gid} = \mathsf{SubsetToId}(S)$ and sends $(\mathsf{select\text{-}group}, \mathsf{sid}, \mathsf{cid}, \mathsf{ssid}, \mathsf{gid})$ to $\mathcal{F}_{\text{BC}_2}$.
(8) **BitVMX execution:** An operator O_j, or any party, sends to $\mathcal{F}_{\text{BitVMX}}$:

$$(\mathsf{claim}, \mathsf{sid}, \mathsf{ssid}, \mathsf{stmt}, w)$$

   where $\mathsf{stmt} = (\mathsf{ssid}$ was burned on $\mathsf{BC}_2, \mathsf{gid}$ was selected ssid corresponds to the locked UTXO on $\mathsf{BC}_1)$ and w is the witness (burn transaction, group selection transaction, inclusion proofs).
(9) $\mathcal{F}_{\text{BitVMX}}$ processes the claim. If valid and unchallenged (or challenges fail): $\mathcal{F}_{\text{BitVMX}}$ outputs $(\mathsf{finalized}, \mathsf{sid}, \mathsf{ssid}, \mathsf{accepted})$.
(10) Upon acceptance: the presigned transaction tx_S is executed on $\mathcal{F}_{\text{BC}_1}$, sending the locked UTXO to $\mathsf{Addr}(\mathsf{pk}_S)$.
(11) **Final spending:** Θ_1 computes $\mathsf{sk}_S = \sum_{j \in S} \mathsf{sh}_j$ and produces a standard Schnorr signature σ under pk_S using sk_S locally. The unforgeability of σ follows from Lemma 4.4. Θ_1 submits $(\mathsf{tx}_{\mathsf{final}}, \sigma)$ to $\mathcal{F}_{\text{BC}_1}$, transferring the asset to any address of Θ_1's choice.

Remark 4.20 (Verification in step U5)**.** The verification $\mathsf{sh}'_i \cdot G = a_i \cdot K_i$ is the key step that makes the protocol work. Since a_i and K_i are public (established during setup), Θ_1 can independently verify each share without trusting the operator. This check detects both invalid shares (from corrupted operators) and transmission errors.

4.6. **Simulator Construction.** We construct a simulator $\mathcal{S}$ for the main UC theorem. $\mathcal{S}$ interacts with the ideal functionality $\mathcal{F}_{\text{TOOP}}$ (receiving leakage) and must produce a view for the environment $\mathcal{Z}$ that is indistinguishable from the real-world view.

4.6.1. *Corruption state.* Let $\mathcal{C} \subset \mathcal{O}$ with $|\mathcal{C}| \leq n-1$ be the set of corrupted operators, known to $\mathcal{S}$ at the start (static corruption). Let $O_j \in \mathcal{O} \setminus \mathcal{C}$ be an honest operator. The owners Θ_0, Θ_1 may be honest or corrupted.

4.6.2. *Simulator description.*

Construction 4.21 (Simulator $\mathcal{S}$)**.** $\mathcal{S}$ maintains the following simulated state:

- For each corrupted $O_i \in \mathcal{C}$: the real key pair (k_i, K_i) and share $\mathsf{sh}_i = a_i k_i$ (obtained from the corruption).
- For each honest $O_i \notin \mathcal{C}$: a simulated public key $\tilde{K}_i$ (generated during setup simulation) and the aggregation coefficient a_i.

Phase 1: Setup simulation.

(1) For each honest operator $O_i \notin \mathcal{C}$:
   - $\mathcal{S}$ samples $\tilde{k}_i \xleftarrow{\$} \mathbb{Z}_q$, computes $\tilde{K}_i = \tilde{k}_i \cdot G$.
   - $\mathcal{S}$ computes $a_i = H_{\text{agg}}(\tilde{K}_i, \{\tilde{K}_1, \ldots, \tilde{K}_n\})$ where corrupted operators' keys are real.
   - $\mathcal{S}$ stores $\widetilde{\mathsf{sh}}_i = a_i \cdot \tilde{k}_i$.

(2) For corrupted operators: $\mathcal{S}$ receives their keys from $\mathcal{Z}$ (modeling the adversary's key choice, subject to rogue-key protection).

(3) $\mathcal{S}$ computes all aggregate public keys pk_S for all non-empty $S \subseteq \mathcal{O}$.

(4) $\mathcal{S}$ simulates the presigned transaction set consistently.

Phase 2: Lock simulation.

(1) Upon receiving $(\mathsf{lock\text{-}notification}, \mathsf{sid}, \mathsf{ssid}, \mathsf{utxo}, \mathsf{addr}_{\mathsf{BC}_2})$ from $\mathcal{F}_{\text{TOOP}}$:
   - If Θ_0 is honest: $\mathcal{S}$ simulates the lock transaction $\mathsf{tx}_{\mathsf{lock}}$ on the simulated BC_1 ledger. The simulated transaction sends utxo to $\mathsf{addr}_{\mathcal{O}}$.
   - If Θ_0 is corrupted: $\mathcal{S}$ receives the lock transaction from $\mathcal{Z}$ and forwards it to $\mathcal{F}_{\text{TOOP}}$. $\mathcal{S}$ checks asset validity (the honest operator's verification step) and instructs $\mathcal{F}_{\text{TOOP}}$ to accept or reject accordingly.

(2) $\mathcal{S}$ simulates operator responses (verification, setup participation).

(3) $\mathcal{S}$ simulates the minting on BC_2.

Phase 3: Transfer simulation.

For wrapped-asset transfers on BC_2 we have: if Θ_{from} is honest, then $\mathcal{S}$ receives $(\mathsf{transfer\text{-}notification}, \mathsf{sid}, \mathsf{ssid}, \Theta_{\text{to}})$ from $\mathcal{F}_{\text{TOOP}}$ and simulates the corresponding BC_2 transfer transaction on the simulated ledger for $\mathcal{Z}$'s view. If Θ_{from} is corrupted, $\mathcal{S}$ receives the transfer transaction from $\mathcal{Z}$, extracts $(\mathsf{ssid}, \Theta_{\text{from}}, \Theta_{\text{to}})$, and forwards $(\mathsf{transfer}_{\mathsf{BC}_2}, \mathsf{sid}, \mathsf{ssid}, \Theta_{\text{from}}, \Theta_{\text{to}})$ to $\mathcal{F}_{\text{TOOP}}$.

Phase 4: Unlock simulation (core challenge).

$\mathcal{S}$ must simulate the ECIES ciphertexts of honest operators' shares without knowing the real shares (in the ideal world, the ideal functionality does not reveal honest operators' shares to $\mathcal{S}$ when Θ_1 is honest).

(1) Upon receiving $(\mathsf{burn\text{-}notification}, \mathsf{sid}, \mathsf{ssid}, \widetilde{\mathsf{pk}}_1)$ from $\mathcal{F}_{\text{TOOP}}$:
   - $\mathcal{S}$ simulates the burn transaction on the simulated BC_2 ledger.

(2) **Simulating honest operators' share delivery (case: Θ_1 honest):**
- For each honest $O_i \notin \mathcal{C}$:
  - $\mathcal{S}$ does *not* know sh_i. However, $\mathcal{S}$ must produce a ciphertext $\tilde{c}_i$ that is indistinguishable from $\mathsf{ECIES.Encrypt}(\widetilde{\mathsf{pk}}_1, \mathsf{sh}_i)$.
  - $\mathcal{S}$ produces $\tilde{c}_i \leftarrow \mathsf{ECIES.Encrypt}(\widetilde{\mathsf{pk}}_1, 0^{|\mathsf{sh}_i|})$ - i.e., encrypts a dummy value of the same length.
  - By the IND-CPA security of ECIES (under the ECDHP assumption), $\tilde{c}_i$ is computationally indistinguishable from the real ciphertext.
- $\mathcal{S}$ delivers $(\tilde{c}_i, \tilde{K}_i)$ to the simulated view for Θ_1.

(3) **Simulating honest operators' share delivery (case: Θ_1 corrupted):**
- Since Θ_1 is corrupted, $\mathcal{S}$ controls Θ_1 and knows $\widetilde{\mathsf{sk}}_1$. From $\mathcal{F}_{\text{TOOP}}$, $\mathcal{S}$ receives the shares of honest operators (the functionality reveals shares to corrupted recipients). $\mathcal{S}$ encrypts them honestly.

(4) **Simulating corrupted operators' share delivery:**
- $\mathcal{S}$ receives (c_i, sh_i) from $\mathcal{Z}$ for each corrupted $O_i \in \mathcal{C}$.
- $\mathcal{S}$ forwards these to $\mathcal{F}_{\text{TOOP}}$ and to Θ_1's simulated view.

(5) **Group selection simulation:**
- If Θ_1 is honest: $\mathcal{S}$ receives $(\mathsf{group\text{-}selected}, \mathsf{sid}, \mathsf{ssid}, \mathsf{gid})$ from $\mathcal{F}_{\text{TOOP}}$ and simulates the corresponding BC_2 transaction.
- If Θ_1 is corrupted: $\mathcal{S}$ receives gid from $\mathcal{Z}$ and forwards to $\mathcal{F}_{\text{TOOP}}$.

(6) **BitVMX claim simulation:**
- $\mathcal{S}$ simulates the BitVMX claim submission with the correct statement and witness.
- If corrupted operators attempt to challenge a valid claim: $\mathcal{S}$ simulates the challenge-response protocol. By the completeness of $\mathcal{F}_{\text{BitVMX}}$, valid claims are accepted.
- If an operator submits a false claim (no valid burn occurred): the honest operator (simulated by $\mathcal{S}$) challenges the claim. By the soundness of $\mathcal{F}_{\text{BitVMX}}$, the false claim is rejected.

(7) **Unlock completion simulation:**
- $\mathcal{S}$ simulates the presigned transaction tx_S being included on BC_1.
- $\mathcal{S}$ simulates the final spending transaction by Θ_1.

Phase 5: Reclaim simulation.

(1) If Θ_0 is honest: upon receiving $(\text{reclaim}, \text{sid}, \text{ssid})$ from $\mathcal{F}_{\text{TOOP}}$, $\mathcal{S}$ simulates the timelock-expiry transaction on the simulated BC_1 ledger, returning the UTXO to Θ_0's address.

(2) If Θ_0 is corrupted: $\mathcal{S}$ receives the reclaim transaction from $\mathcal{Z}$, verifies the timelock has expired and no minting has occurred, and forwards $(\text{reclaim}, \text{sid}, \text{ssid})$ to $\mathcal{F}_{\text{TOOP}}$.

4.6.3. *Indistinguishability argument overview.* The indistinguishability between the real and ideal worlds proceeds through a sequence of hybrid games. We outline the structure here; the full proof appears in Section 4.7.

(1) **Real world.** All parties execute Π_{TOOP} with real sub-protocols for MuSig2, ECIES, BitVMX, BC_1, BC_2.
(2) **Replace sub-protocols with ideal functionalities.** By the UC composition theorem, this is indistinguishable if each sub-protocol UC-realizes its functionality. Here we use the known UC-security results for Schnorr, ECIES, and the Bitcoin ledger, plus our assumed $\mathcal{F}_{\mathrm{BitVMX}}$ realization.
(3) **Replace honest operators' shares with dummy ciphertexts.** For each honest operator O_i (when Θ_1 is honest), replace $\mathsf{ECIES.Encrypt}(\widetilde{\mathsf{pk}}_1, \mathsf{sh}_i)$ with $\mathsf{ECIES.Encrypt}(\widetilde{\mathsf{pk}}_1, 0^{|\mathsf{sh}_i|})$. By IND-CPA security of ECIES, each replacement changes the environment's distinguishing advantage by at most $\mathsf{negl}(\kappa)$. Since there are at most n operators (polynomial), the total distinguishing advantage is at most $n \cdot \mathsf{negl}(\kappa) = \mathsf{negl}(\kappa)$.
(4) **Ideal world.** The resulting execution is exactly the ideal-world execution with $\mathcal{F}_{\mathrm{TOOP}}$ and simulator $\mathcal{S}$.

4.7. **Main Theorem and Proof.**

Theorem 4.22 (Main UC Theorem)**.** *Under the discrete logarithm assumption in $E(\mathbb{Z}_q)$, the random oracle model for H_{agg} and H_{sig}, the ECDHP assumption, the IND-CPA security of the symmetric component of ECIES, and assuming $\mathcal{F}_{\mathrm{BitVMX}}$ is realized by the BitVMX system, the protocol Π_{TOOP} UC-realizes the ideal functionality $\mathcal{F}_{\mathrm{TOOP}}$ in the $(\mathcal{F}_{\mathrm{MuSig}}, \mathcal{F}_{\mathrm{ECIES}}, \mathcal{F}_{\mathrm{BitVMX}}, \mathcal{F}_{\mathrm{BC}_1}, \mathcal{F}_{\mathrm{BC}_2})$-hybrid world against static adversaries corrupting at most $n-1$ operators, where the owners Θ_0 and Θ_1 may be independently honest or corrupted.*

Proof. We must show that for every PPT adversary $\mathcal{A}$ there exists a PPT simulator $\mathcal{S}$ (Construction 4.21) such that for all PPT environments $\mathcal{Z}$:

$$\mathsf{IDEAL}_{\mathcal{F}_{\mathrm{TOOP}},\mathcal{S},\mathcal{Z}}(\kappa, z) \approx_c \mathsf{HYBRID}^{\mathcal{F}_{\mathrm{MuSig}},\mathcal{F}_{\mathrm{ECIES}},\mathcal{F}_{\mathrm{BitVMX}},\mathcal{F}_{\mathrm{BC}_1},\mathcal{F}_{\mathrm{BC}_2}}_{\Pi_{\mathrm{TOOP}},\mathcal{A},\mathcal{Z}}(\kappa, z).$$

We proceed through a sequence of hybrid experiments.

Hybrid H_0 (Real hybrid-world execution): This is the execution

$$\mathsf{HYBRID}^{\mathcal{F}_{\mathrm{MuSig}},\mathcal{F}_{\mathrm{ECIES}},\mathcal{F}_{\mathrm{BitVMX}},\mathcal{F}_{\mathrm{BC}_1},\mathcal{F}_{\mathrm{BC}_2}}_{\Pi_{\mathrm{TOOP}},\mathcal{A},\mathcal{Z}}.$$

All honest parties follow the protocol Π_{TOOP} with real key material.

Hybrid H_1 (Introduce the simulator's setup): Replace the honest operators' key generation with $\mathcal{S}$'s simulated key generation. Since k_i (in H_0) and $\tilde{k}_i$ (in H_1) are sampled uniformly and independently from $\mathbb{Z}_q$, the distribution $(K_1, \ldots, K_n, a_1, \ldots, a_n)$ is identical in H_0 and H_1 (the aggregation coefficients are deterministic functions of the public keys). Therefore $H_1 \equiv H_0$.

Hybrid H_2 (Replace honest shares with dummies for honest Θ_1): For each honest operator $O_i \notin \mathcal{C}$, when Θ_1 is honest, replace the ECIES ciphertext:

$$c_i = \mathsf{ECIES.Encrypt}(\widetilde{\mathsf{pk}}_1, \mathsf{sh}_i) \quad \longrightarrow \quad \tilde{c}_i = \mathsf{ECIES.Encrypt}(\widetilde{\mathsf{pk}}_1, 0^{|\mathsf{sh}_i|}).$$

We do this one operator at a time. Let $H_2^{(0)} = H_1$ and $H_2^{(\ell)}$ be the hybrid where the first ℓ honest operators' ciphertexts have been replaced.

Claim 4.23. For each $\ell \in \{0, \ldots, |\mathcal{O} \setminus \mathcal{C}| - 1\}$:

$$\left|\Pr[\mathcal{Z} \text{ outputs 1 in } H_2^{(\ell+1)}] - \Pr[\mathcal{Z} \text{ outputs 1 in } H_2^{(\ell)}]\right| \leq \mathsf{Adv}_{\text{ECIES}}^{\text{IND-CPA}}(\kappa).$$

Proof of Claim 4.23. Suppose $\mathcal{Z}$ distinguishes $H_2^{(\ell)}$ from $H_2^{(\ell+1)}$ with non-negligible advantage. We construct an IND-CPA adversary $\mathcal{B}$ against ECIES:

(1) $\mathcal{B}$ receives the ECIES public key $\widetilde{\mathsf{pk}}_1$ from its IND-CPA challenger (this is Θ_1's encryption key; Θ_1 is honest, so $\mathcal{B}$ does not know $\widetilde{\mathsf{sk}}_1$).
(2) $\mathcal{B}$ runs the full hybrid experiment $H_2^{(\ell)}$ honestly, except for operator $O_{i_{\ell+1}}$ (the $(\ell+1)$-th honest operator):
   - For the first ℓ honest operators: encrypt dummy values (as in $H_2^{(\ell)}$).
   - For $O_{i_{\ell+1}}$: submit $(m_0, m_1) = (\mathsf{sh}_{i_{\ell+1}}, 0^{|\mathsf{sh}_{i_{\ell+1}}|})$ to the IND-CPA challenger. Receive challenge ciphertext c^*.
   - For remaining honest operators: encrypt real shares.
(3) $\mathcal{B}$ completes the simulation and outputs whatever $\mathcal{Z}$ outputs.
(4) If the challenger encrypted m_0, this is $H_2^{(\ell)}$. If m_1, this is $H_2^{(\ell+1)}$.

Therefore $\mathcal{B}$'s IND-CPA advantage equals $\mathcal{Z}$'s distinguishing advantage between $H_2^{(\ell)}$ and $H_2^{(\ell+1)}$. Corrupted operators may send ciphertexts c^* not produced through $\mathcal{F}_{\text{ECIES}}$. For honest Θ_1, decryption of $c^* \notin$ DecTable invokes real ECIES decryption with $\widetilde{\mathsf{sk}}_1$, which $\mathcal{B}$ does not possess. However, Θ_1 verifies each decrypted value by checking $\mathsf{sh}_i' \cdot G = a_i \cdot K_i$. A ciphertext not produced by honest encryption of sh_i will, upon decryption, yield a value uniformly distributed over $\mathbb{Z}_q$ (by the pseudorandomness of ECIES decryption on invalid ciphertexts), passing verification with probability $1/q = \mathsf{negl}(\kappa)$. Therefore $\mathcal{B}$ can simulate honest-Θ_1 behavior by having Θ_1 discard all shares from corrupted operators whose ciphertexts are not in the DecTable, incurring at most $n \cdot q^{-1} = \mathsf{negl}(\kappa)$ simulation error. □

By a union bound over at most n honest operators:

$$|\Pr[\mathcal{Z} = 1 \text{ in } H_2] - \Pr[\mathcal{Z} = 1 \text{ in } H_1]| \leq n \cdot \mathsf{Adv}_{\text{ECIES}}^{\text{IND-CPA}}(\kappa) = \mathsf{negl}(\kappa).$$

Remark 4.24. A subtlety in the $H_1 \to H_2$ transition requires explicit justification. In H_1, honest Θ_1 decrypts each ciphertext c_i via $\mathcal{F}_{\text{ECIES}}$ and verifies $\mathsf{sh}_i' \cdot G = a_i \cdot K_i$. In H_2, the ciphertexts of honest operators are replaced with encryptions of $0^{|\mathsf{sh}_i|}$. One might worry that Θ_1's verification would now fail (since $0 \cdot G \neq a_i \cdot K_i$). However, since we are in the $\mathcal{F}_{\text{ECIES}}$-hybrid world, honest Θ_1's decryption goes through $\mathcal{F}_{\text{ECIES}}$'s `DecTable`: in both H_1 and H_2, the honest operator sends $(\mathsf{encrypt}, \mathsf{sid}, \mathsf{pk}_{f_1}, \mathsf{sh}_i)$ to $\mathcal{F}_{\mathsf{ECIES}}$, which stores $\mathsf{DecTable}[c_i] \leftarrow \mathsf{sh}_i$. The game hop from H_1 to H_2 modifies the ciphertext computation inside $\mathcal{F}_{\mathsf{ECIES}}$: in H_2, $\mathcal{F}_{\mathsf{ECIES}}$ computes $\tilde{c}_i \leftarrow \mathsf{ECIES.Encrypt}(\mathsf{pk}_{f_1}, 0^{|\mathsf{sh}_i|})$ instead of $c_i \leftarrow \mathsf{ECIES.Encrypt}(\mathsf{pk}_{f_1}, \mathsf{sh}_i)$, while the $\mathsf{DecTable}$ mapping remains $\mathsf{DecTable}[\tilde{c}_i] \leftarrow \mathsf{sh}_i$. Upon decryption, honest Θ_1 retrieves sh_i from $\mathsf{DecTable}$ regardless of the ciphertext content. The IND-CPA replacement thus affects only the adversary's view of the ciphertexts on the network.

Claim 4.25. All ideal-world invariants (I1)–(I6) of $\mathcal{F}_{\text{TOOP}}$ are satisfied in H_2 with overwhelming probability. The invariants are verified as properties of H_2 and do not require an additional hybrid step. However, their proofs rely on the computational

assumptions underlying H_2 (DLP, EUF-CMA, $\mathcal{F}_{\text{BitVMX}}$ soundness), which bound the probability of violation.

Proof of Claim 4.25. We verify each invariant by showing that any violation implies a break of an underlying assumption.

(1) **Asset validity violation:** Suppose $\mathcal{A}$ (via a corrupted Θ_0) locks a UTXO not owned by Θ_0. This requires producing a valid signature for the UTXO's spending condition. By the existential unforgeability of Schnorr signatures (DLP assumption), this occurs with negligible probability. Moreover, the honest operator verifies ownership before proceeding, providing a second barrier.
(2) **Transfer authenticity violation (mint without lock):** The honest operator only triggers minting after verifying the lock on $\mathcal{F}_{\text{BC}_1}$. A mint without a corresponding lock requires either: (i) forging a Bitcoin transaction (breaking $\mathcal{F}_{\text{BC}_1}$'s validity), or (ii) all operators being corrupted (contradicts $|\mathcal{C}| \leq n-1$).
(3) **Transfer authenticity violation (unlock without burn):** The BitVMX claim for unlocking includes as its statement that a burn occurred on BC_2. If no burn occurred, the statement is false. By the computational soundness of $\mathcal{F}_{\text{BitVMX}}$, the honest operator challenges the claim, and the false claim is rejected with overwhelming probability.
(4) **Double-spend prevention:** The UTXO model of $\mathcal{F}_{\text{BC}_1}$ enforces that each UTXO can be spent exactly once. The smart contract state on $\mathcal{F}_{\text{BC}_2}$ enforces that each wrapped asset can be burned at most once. The honest operator maintains consistent state and refuses to participate in duplicate operations.
(5) **Ownership transfer security violation:** Suppose $\mathcal{A}$ (not knowing $\widetilde{\mathsf{sk}}_1$) redirects the transfer. $\mathcal{A}$ must either:
   - Decrypt honest operators' ECIES ciphertexts without $\widetilde{\mathsf{sk}}_1$: contradicts ECDHP assumption (by Claim 4.23's reduction).
   - Reconstruct sk_S for a subset S containing the honest operator O_j without sh_j: contradicts the DLP assumption (by Lemma 4.3).
   - Use a subset $S' \subseteq \mathcal{C}$ (only corrupted operators): $\mathcal{A}$ knows all shares for S'. However, the group-selection transaction on BC_2 is restricted to the party that submitted the burn ($\mathcal{F}_{BC_2}$, Interface 3 accepts select-group only from Θ_1). Since Θ_1 is honest and $\mathcal{A}$ does not know Θ_1's BC_2 signing key, $\mathcal{A}$ cannot forge a group-selection transaction. Thus $\mathcal{A}$ cannot cause the BitVMX program to authorize a spend to $\mathsf{pk}_{S'}$ for $S' \neq S^*$. The honest operator O_j challenges any BitVMX claim whose statement does not match Θ_1's legitimate burn and group selection.
(6) **Liveness:** The honest operator O_j ensures forward progress at each phase:
   - Lock verification: within $\Delta_{\mathsf{lock}} = k_1 \cdot \tau_1$ after $\mathsf{tx}_{\mathsf{lock}}$ confirmation.
   - Minting: within $\Delta_{\mathsf{mint}} = k_2 \cdot \tau_2$ after lock verification.
   - Share delivery and verification: within $\Delta_{\text{share}} = 2\Delta$ after burn detection (the protocol requires Θ_1 to wait 2Δ to ensure all honest operators' shares have been received).
   - BitVMX execution: within $\Delta_{\mathsf{unlock}} = \Delta_{\mathsf{chal}} + k_1 \cdot \tau_1$ after group selection.

   Total: $\Delta_{\mathsf{total}} = \Delta_{\mathsf{lock}} + \Delta_{\mathsf{mint}} + \Delta_{\mathsf{burn}} + \Delta_{\mathsf{share}} + \Delta_{\mathsf{unlock}}$ where $\Delta_{\mathsf{burn}} = k_2 \cdot \tau_2$ (burn confirmation time). Corrupted operators cannot prevent the honest operator

from executing these steps (they operate through $\mathcal{F}_{\mathrm{BC}_1}$, $\mathcal{F}_{\mathrm{BC}_2}$, $\mathcal{F}_{\mathrm{BitVMX}}$ which guarantee inclusion within bounded time).

This establishes Claim 4.25. □

H_2 is the ideal world. By Claim 4.23, the adversary's view in H_2 is computationally indistinguishable from $H_1 \equiv H_0$ (the real hybrid-world execution). By Claim 4.25, all ideal-world invariants hold in H_2. The simulator $\mathcal{S}$ (Construction 4.21) produces all messages visible to $\mathcal{Z}$ in H_2. We verify that H_2 is functionally identical to $\mathsf{IDEAL}_{\mathcal{F}_{\mathrm{TOOP}},\mathcal{S},\mathcal{Z}}$ by checking each interface of $\mathcal{F}_{\mathrm{TOOP}}$: (1) Lock/Mint: $\mathcal{S}$ simulates BC_1/BC_2 ledger transactions (Phase 2); (2) Burn/ShareDist: honest operators' ciphertexts are dummy (Claim 4.23), corrupted operators' ciphertexts are forwarded from $\mathcal{Z}$ (Phase 3, step 4); (3) GroupSel/Unlock: $\mathcal{S}$ simulates BC_2 group-selection and BC_1 presigned transactions; for honest Θ_1, $\mathcal{S}$ controls the simulated $\mathcal{F}_{\mathsf{BC}_1}$ and can include the final spending transaction without knowing sk_S; (4) Reclaim: $\mathcal{S}$ simulates the timelock-expiry transaction. In all cases, $\mathcal{S}$'s output distribution matches H_2:

- Honest operators' shares are hidden (dummy ciphertexts - Claim 4.23).
- All security invariants (I1)-(I6) are enforced (Claim 4.25).
- $\mathcal{S}$ produces the simulated view consistently.

Conclusion. Combining the hybrid steps:

$$|\Pr[\mathcal{Z} = 1 \text{ in } \mathsf{IDEAL}] - \Pr[\mathcal{Z} = 1 \text{ in } \mathsf{HYBRID}]| \leq n \cdot \mathsf{Adv}^{\text{IND-CPA}}_{\text{ECIES}}(\kappa) + \epsilon_{\text{inv}}(\kappa),$$

where $\epsilon_{\text{inv}}(\kappa)$ bounds the probability that any invariant (I1)-(I6) is violated in H_2 (Claim 4.25). This term absorbs: the DLP advantage (asset validity, Lemma 4.3), the EUF-CMA advantage (transfer authenticity via Schnorr unforgeability), and the $\mathcal{F}_{\mathrm{BitVMX}}$ soundness guarantee (double-spend prevention). Since n is polynomial in κ, $\mathsf{Adv}^{\text{IND-CPA}}_{\text{ECIES}}(\kappa)$ is negligible by assumption, and $\epsilon_{\text{inv}}(\kappa)$ is negligible as argued in Claim 4.25, the total distinguishing advantage is negligible. This completes the proof. □

Corollary 4.26 (Composition)**.** *Let π_{MuSig}, π_{ECIES}, π_{BitVMX}, π_{BC_1}, π_{BC_2} be protocols that UC-realize $\mathcal{F}_{\mathrm{MuSig}}$, $\mathcal{F}_{\mathrm{ECIES}}$, $\mathcal{F}_{\mathrm{BitVMX}}$, $\mathcal{F}_{\mathrm{BC}_1}$, $\mathcal{F}_{\mathrm{BC}_2}$ respectively. Then the composed protocol $\Pi^{\mathrm{comp}}_{\mathrm{TOOP}}$—obtained by replacing each ideal-functionality call in Π_{TOOP} with the corresponding real sub-protocol—UC-realizes $\mathcal{F}_{\mathrm{TOOP}}$ in the plain model.*

Proof. Π_{TOOP} UC-realizes $\mathcal{F}_{\mathrm{TOOP}}$ in the $(\mathcal{F}_{\mathrm{MuSig}}, \mathcal{F}_{\mathrm{ECIES}}, \mathcal{F}_{\mathrm{BitVMX}}, \mathcal{F}_{\mathrm{BC}_1}, \mathcal{F}_{\mathrm{BC}_2})$-hybrid world by Theorem 4.22. The UC composition theorem [6] states that if a protocol π UC-realizes $\mathcal{F}$ in the $\mathcal{G}$-hybrid world, and protocol ρ UC-realizes $\mathcal{G}$, then the composed protocol π^{ρ} (where calls to $\mathcal{G}$ are replaced by executions of ρ) UC-realizes $\mathcal{F}$ in the plain model. Applying this iteratively to each sub-protocol—using the MuSig2 UC-realization of [4], the standard ECIES UC-realization under ECDHP, the assumed realization of $\mathcal{F}_{\mathrm{BitVMX}}$, and the ledger realizations of [3]—yields theresult. Since the five sub-functionalities have disjoint session identifiers, the iterative application is well-defined. □

4.8. **Deriving Game-Based Properties.** The ideal functionality $\mathcal{F}_{\mathrm{TOOP}}$ directly implies all game-based security properties (Definitions 2.2, 2.3, 2.4, 2.6, 2.9, and 2.11). The 1-out-of-n guarantee (Definition 2.10) follows from their simultaneous validity under the corruption bound $|\mathcal{C}| \leq n - 1$. Each property follows from the corresponding invariant of $\mathcal{F}_{\mathrm{TOOP}}$.

Corollary 4.27 (Asset Validity - cf. Definition 2.2)**.** *Under the assumptions of Theorem 4.22, the protocol satisfies asset validity:*

$$\Pr[\mathsf{Lock}(X, \Theta_0) \text{ accepted} \mid X \not\subseteq \mathsf{Assets}(\Theta_0, \mathsf{BC}_1)] \leq \mathsf{negl}(\kappa).$$

Proof. In the ideal world, $\mathcal{F}_{\text{TOOP}}$ checks $\mathsf{utxo} \in \mathsf{Assets}(\Theta_0, \mathsf{BC}_1)$ before accepting any Lock request (Interface 1, asset validity check). Any environment $\mathcal{Z}$ that causes an invalid lock to be accepted in the real world would distinguish real from ideal, contradicting Theorem 4.22. □

Corollary 4.28 (Transfer Authenticity - cf. Definition 2.3)**.** *The protocol satisfies transfer authenticity: no wrapped asset is created without a corresponding lock, and no UTXO is unlocked without a corresponding burn, except with negligible probability.*

Proof. $\mathcal{F}_{\text{TOOP}}$'s Mint interface (Interface 2) requires $\mathsf{LockedAssets}[\mathsf{ssid}].\mathsf{status} = \mathsf{locked}$. $\mathcal{F}_{\text{TOOP}}$'s Unlock interface (Interface 7) requires $\mathsf{BurnRecords}[\mathsf{ssid}] \neq \bot$. Violation in the real world implies distinguishing advantage against Theorem 4.22. □

Corollary 4.29 (Double-Spend Prevention - cf. Definition 2.4)**.** *The protocol prevents double-spending with overwhelming probability.*

Proof. $\mathcal{F}_{\text{TOOP}}$'s Burn interface (Interface 4) enforces $\mathsf{ssid} \notin \mathsf{BurnRecords}$ (each asset burned at most once). $\mathcal{F}_{\text{TOOP}}$'s Unlock interface moves the UTXO via $\mathcal{F}_{\text{BC}_1}$, which enforces single-spending of UTXOs. The Completed set ensures each ssid is processed at most once. □

Corollary 4.30 (Ownership Transfer Security - cf. Definition 2.6)**.** *For any adversary $\mathcal{A}$ not knowing Θ_1's secret key, the probability that $\mathcal{A}$ redirects the transfer away from Θ_1 is negligible.*

Proof. The argument proceeds in three steps:

(1) In the ideal world, $\mathcal{F}_{\text{TOOP}}$'s invariant (I5) guarantees that $\mathcal{S}$ receives no information about honest operators' shares when Θ_1 is honest.
(2) By Theorem 4.22, no PPT environment $\mathcal{Z}$ can distinguish the real-world execution from the ideal-world execution with non-negligible advantage.
(3) Therefore, in the real world, no PPT adversary $\mathcal{A}$ can extract honest operators' shares with non-negligible probability. If $\mathcal{A}$ could extract an honest operator's share with non-negligible probability ϵ, we construct $\mathcal{Z}$ as follows: $\mathcal{Z}$ runs $\mathcal{A}$ internally, provides $\mathcal{A}$ with the real/ideal view, and when $\mathcal{A}$ outputs a candidate share sh'_j, $\mathcal{Z}$ checks $\mathsf{sh}'_j \cdot G \stackrel{?}{=} a_j \cdot K_j$. If the check passes, $\mathcal{Z}$ outputs 1; otherwise 0. In the real world, $\mathcal{A}$ succeeds with probability ϵ; in the ideal world (dummy ciphertexts), the share is information-theoretically hidden from $\mathcal{A}$, so $\mathcal{A}$ succeeds with probability at most $1/q = \text{negl}(\kappa)$. Thus $\mathcal{Z}$'s distinguishing advantage is $\epsilon - 1/q$, contradicting (2) if ϵ is non-negligible.

By Lemma 4.3, without the honest operator's share, $\mathcal{A}$ cannot reconstruct sk_S for any subset S containing the honest operator. Redirecting the transfer would therefore require either breaking ECDHP (to decrypt the honest share) or DLP (to reconstruct sk_S without it), both negligible. □

Corollary 4.31 (Liveness - cf. Definition 2.9)**.** *If Θ_1 is honest and at least one operator is honest, the transfer completes within bounded time Δ_{total}.*

Proof. $\mathcal{F}_{\text{TOOP}}$'s liveness invariant (I6) guarantees completion within Δ_{total}, provided $\mathcal{S}$ cannot delay beyond the specified bounds. In the proof of Theorem 4.22 (Claim 4.25, invariant I6), the honest operator ensures forward progress at each phase, with concrete time bounds derived from the timing guarantees of $\mathcal{F}_{\text{BC}_1}$, $\mathcal{F}_{\text{BC}_2}$, and $\mathcal{F}_{\text{BitVMX}}$. □

Corollary 4.32 (1-out-of-n Security - cf. Definition 2.10)**.** *All security properties hold simultaneously when at most $n-1$ operators are corrupted.*

Proof. Theorem 4.22 establishes UC-realization under the corruption bound $|\mathcal{C}| \leq n-1$. Since the ideal functionality $\mathcal{F}_{\text{TOOP}}$ enforces all invariants (I1)–(I6) simultaneously, all game-based properties hold simultaneously as corollaries. □

Corollary 4.33 (Key Share Confidentiality - cf. Definition 2.11)**.** *For any adversary not knowing Θ_1's private key, the probability of extracting any honest operator's key share from the encrypted communications is negligible.*

Proof. This is directly captured by $\mathcal{F}_{\text{TOOP}}$'s invariant (I5): when Θ_1 is honest, $\mathcal{S}$ receives no information about honest operators' shares beyond delivery confirmation. The indistinguishability follows from the IND-CPA security of ECIES (Claim 4.23 in the main proof). Note that the UC analysis provides a strictly stronger guarantee than Definition 2.11: invariant (I5) gives semantic security (indistinguishability of ciphertexts), whereas Definition 2.11 guarantees only key-recovery resistance. Definition 2.11 should be understood as a minimal baseline; the UC formulation is the authoritative security statement. □

5. Conclusion

We have presented TOOP, a cross-chain protocol that transfers ownership of Bitcoin UTXOs to addresses undetermined at locking time, and established its security through a full UC analysis. The main theorem (Theorem 4.22) shows that Π_{TOOP} UC-realizes $\mathcal{F}_{\text{TOOP}}$ under DLP, ECDHP, IND-CPA, and ROM assumptions, with the UC composition theorem guaranteeing preservation under arbitrary concurrent composition. All six game-based security properties are recovered as corollaries.

6. Future research

This protocol involves several trade-offs related to scalability, communication overhead, and resilience against specific attack scenarios. These trade-offs, discussed below, also help define key directions for our future research.

6.1. **Scalability.** This protocol is designed with fixed on-chain costs, regardless of the number of operators involved. However, it faces challenges in managing computational complexity and communication overhead as the number of operators n increases. Indeed, concerning signature generation complexity, the protocol requires signature generation for each subset of operators, resulting in a total of $2^n - 1$ signatures for n operators (one per non-empty subset). This exponential growth limits practical application to scenarios where $n < 30$. While n remains small in most practical scenarios, the exponential growth could become a bottleneck in larger systems.
There exist several options to tackle the problem with the exponential growth in signature calculations, being one of them key aggregation; key can be aggregated into

groups of up to a fixed size $N < n$. We observe that with this approach two strategies are possible:

(1) Partition into disjoint groups: Partition n operators into $\lfloor n/N \rfloor$ disjoint groups of size N. Enumerate subsets within each group: complexity $\lfloor n/N \rfloor \cdot (2^N - 1)$. This requires $\lfloor n/N \rfloor$ on-chain group signatures.
(2) All subsets of fixed size N: Consider only subsets of size exactly N. Complexity: $\binom{n}{N}$ aggregate public keys and signatures.

When it comes to the communications overhead, communications between operators are on the order of $O(n^2)$, which may become a limiting factor for very large committees. As the number of operators grows, the increase in communication requirements could strain resources and delay operations. In any case, this is the same scalability requirements present in other parts of the system.

6.2. **Attacks.**

Remark 6.1 (Corruption model)**.** The security analysis assumes static corruption (the adversary's corruption choices are fixed before execution). Extending TOOP to dynamic operator sets (operators joining or leaving between sessions) would require moving to the adaptive corruption model, which is significantly more challenging for UC proofs.

This is a protocol designed to ensure security even in the presence of dishonest operators. However, certain attacks can still be attempted by malicious participants. Below, we outline the possible attack vectors and the corresponding mitigation strategies to be explored in the future.

(1) Misleading information to Θ_1: dishonest operators may send invalid or incorrect secret key information to Θ_1 during the unlocking phase. This could lead to delays or errors in the unlocking process.

    Mitigation: Θ_1 verifies all secret keys received against the public keys generated during the lock phase. Any invalid keys are discarded, ensuring that only valid keys are used for the unlocking process.
(2) Refusal to collaborate: dishonest operators may refuse to participate in the unlocking process by withholding their secret keys from Θ_1.

    Mitigation: The protocol only requires one honest operator to participate for the unlocking to succeed. As long as at least one operator follows the protocol, the unlocking process remains functional.
(3) Key sharing with unauthorized users: dishonest operators may share their secret keys with unauthorized users, enabling them to impersonate Θ_1 or gain access to the locked assets.

    Mitigation: Θ_1 independently verifies the validity of received keys. Additionally, since each operator's key share is encrypted specifically for Θ_1 using ECIES, unauthorized users cannot use the keys without compromising the encryption.
(4) Collusion between operators: operators may collude to manipulate the unlocking process by selecting a different group ID or bypassing Θ_1's input. This could potentially allow them to access the locked assets improperly.

Mitigation: An honest operator can challenge any unlocking attempt that deviates from the protocol by leveraging the BitVMX dispute mechanism. This ensures that any unauthorized unlocking attempts are detected and prevented.

Acknowledgements. The authors would like to thank Nicolas Biri, Torben Poguntke, Dimitar Jetchev, Thomas Vellekoop, Jesús Díaz Vico, and Romain Pellerin from Input Output Global, and Martin Jonas, Diego Tiscornia, Emilio García, Hernán Pérez Rodal, and Agustin Zaballa from Fairgate Labs for their insightful contributions to this research.

References

[1] Augusto, André, Rafael Belchior, Miguel Correia, André Vasconcelos, Luyao Zhang, and Thomas Hardjono. "Sok: Security and privacy of blockchain interoperability." In 2024 IEEE Symposium on Security and Privacy (SP), pp. 3840-3865. IEEE, 2024.

[2] Aumayr, Lukas, et al. "BitVM: Quasi-Turing Complete Computation on Bitcoin." Cryptology ePrint Archive (2024).

[3] Badertscher, Christian, Ueli Maurer, Daniel Tschudi, and Vassilis Zikas. "Bitcoin as a Transaction Ledger: A Composable Treatment: C. Badertscher et al." Journal of Cryptology 37, no. 2 (2024): 18.

[4] Baum, Carsten, Bernardo David, Elena Pagnin, and Akira Takahashi. "Universally composable interactive and ordered multi-signatures." In IACR International Conference on Public-Key Cryptography, pp. 3-31. Cham: Springer Nature Switzerland, 2025.

[5] Borkowski, Michael, et al. "DeXTT: Deterministic cross-blockchain token transfers." IEEE access 7 (2019): 111030-111042.

[6] Canetti, Ran. "Universally composable security: A new paradigm for cryptographic protocols." In Proceedings 42nd IEEE Symposium on Foundations of Computer Science, pp. 136-145. IEEE, 2001.

[7] Dangat, Shantanu, et al. "OTEx: Ownership Transfer and Execution Protocol for Blockchain Interoperability." 2024 IEEE International Conference on Blockchain (Blockchain). IEEE, 2024.

[8] Galbraith, Steven D. *Mathematics of public key cryptography.* Cambridge University Press, 2012.

[9] Garoffolo, Alberto, Dmytro Kaidalov, and Roman Oliynykov. "Zendoo: A zk-SNARK verifiable cross-chain transfer protocol enabling decoupled and decentralized sidechains." 2020 IEEE 40th International Conference on Distributed Computing Systems (ICDCS). IEEE, 2020.

[10] van Glabbeek, Rob, Vincent Gramoli, and Pierre Tholoniat. "Cross-chain payment protocols with success guarantees." Distributed Computing 36.2 (2023): 137-157.

[11] Karantias, Kostis, Aggelos Kiayias, and Dionysis Zindros. "Proof-of-burn." Financial Cryptography and Data Security: 24th International Conference, FC 2020, Kota Kinabalu, Malaysia, February 10–14, 2020 Revised Selected Papers 24. Springer International Publishing, 2020.

[12] Network, Kyber, and R. Protocol. "Wrapped Tokens: A Multi-Institutional Framework for Tokenizing Any Asset." Wrapped tokens: A multi-institutional framework for tokenizing any asset",[online] Available: https://wbtc. network/assets/wrapped-tokens-whitepaper. pdf (2022).

[13] Lerner, Sergio Demian, et al. "BitVMX: A CPU for Universal Computation on Bitcoin." arXiv preprint arXiv:2405.06842 (2024).

[14] Linus, Robin, et al. "BitVM2: Bridging Bitcoin to Second Layers." (2024): 2024-11.

[15] Liu, Weiwei, et al. "AucSwap: A Vickrey auction modeled decentralized cross-blockchain asset transfer protocol." Journal of Systems Architecture 117 (2021): 102102.

[16] Nick, Jonas, Andrew Poelstra, and Gregory Sanders. "Liquid: A bitcoin sidechain." Liquid white paper. URL https://blockstream. com/assets/downloads/pdf/liquid-whitepaper. pdf (2020).

[17] Nick, Jonas, Tim Ruffing, and Yannick Seurin. "MuSig2: Simple two-round Schnorr multi-signatures." Annual International Cryptology Conference. Cham: Springer International Publishing, 2021.

[18] Nolan, Tier, "Alt chains and atomic transfers, 2013". https://bitcointalk.org/index.php?topic=193281.0 (accessed May 26, 2024).

[19] Pillai, Babu, et al. "The burn-to-claim cross-blockchain asset transfer protocol." 2020 25th International Conference on Engineering of Complex Computer Systems (ICECCS). IEEE, 2020.
[20] Pointcheval, David, and Jacques Stern. "Security arguments for digital signatures and blind signatures." Journal of cryptology 13, no. 3 (2000): 361-396.
[21] Sober, Michael, et al. "Decentralized cross-blockchain asset transfers with transfer confirmation." Cluster computing 26.4 (2023): 2129-2146.
[22] Wood, Gavin. "Polkadot: Vision for a heterogeneous multi-chain framework." White paper 21.2327 (2016): 4662.
[23] Zamyatin, Alexei, et al. "Xclaim: Trustless, interoperable, cryptocurrency-backed assets." 2019 IEEE symposium on security and privacy (SP). IEEE, 2019.
[24] Zamyatin, A. et al. (2021). SoK: Communication Across Distributed Ledgers. In: Borisov, N., Diaz, C. (eds) Financial Cryptography and Data Security. FC 2021. Lecture Notes in Computer Science(), vol 12675. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-662-64331-0_1

Appendix A. Algorithms

Below follows a description in pseudocode of all the algorithms involved in the transfer of ownership protocol.

Algorithm 7 Generate Operator Powerset

Require: Set of operators $O = \{O_1, O_2, \ldots, O_n\}$ where $n > 1$
Ensure: Powerset $P(O)$ containing all possible subsets of O

function GenCombinations($elems, size$)
  if $size = 0$ **or** $size = |elems|$ **then return** $size = 0?[\emptyset] : [elems]$
  end if
  $combs \leftarrow \emptyset$, $first \leftarrow elems[0]$, $rest \leftarrow elems[1:]$
  for all $smallComb \in$ GenCombinations($rest, size-1$) **do** $combs \leftarrow combs \cup \{[first] + smallComb\}$
  end for
  $combs \leftarrow combs \cup$ GenCombinations($rest, size$) **return** $combs$
end function
function GenOperatorPowerset(ops)
  if $|ops| < 2$ **then return** "Error: Need at least 2 operators"
  end if
  $pset \leftarrow \emptyset$, $opList \leftarrow$ list(ops)
  for $subSize \leftarrow 1$ **to** $|opList|$ **do**
    $pset \leftarrow pset \cup$ GenCombinations($opList, subSize$)
  end forreturn $pset$
end function

Algorithm 8 Key Generation for Locked Assets

Require: Set of locked Bitcoin assets A and operators $O = \{O_1, O_2, \ldots, O_n\}$
Ensure: Dictionary mapping each operator to their key pairs for each asset

function GenKeys($assets, ops, G, q$)
  $aKeys \leftarrow \emptyset$
  for each $a \in assets$ **do**
    $opKeys \leftarrow \emptyset$
    for each $op \in ops$ **do**
      $k_i \leftarrow$ RandSecNum$(1, q-1)$, $K_i \leftarrow$ ScalMult(k_i, G)
      $opKeys[op] \leftarrow \{$"$priv$" $: k_i,$ "pub" $: K_i\}$
    end for
    $aKeys[a] \leftarrow opKeys$
  end forreturn $aKeys$
end function

Algorithm 9 Aggregate Address Generation

Require: Committee of operators $O = \{O_1, O_2, \ldots, O_N\}$, Θ_0
Ensure: Aggregate Bitcoin address and shared operator keys

function GenAggAddr(ops, own_0)
  $opKeys \leftarrow \emptyset$ ▷ Phase 1: Initial Key Generation
  for each $op \in ops$ **do**
    $secKey \leftarrow$ GenSecRand, $pubKey \leftarrow$ GenPubKey($secKey$)
    $opKeys[op] \leftarrow \{"sec" : secKey, "pub" : pubKey\}$
  end for
  $allPubKeys \leftarrow [keys["pub"]$ for $keys$ in $opKeys.values()]$
  $aggPubKey \leftarrow$ CompAggKey($allPubKeys$)
  $aggAddr \leftarrow$ PubKeyToAddr($aggPubKey$)
  $ownInfo \leftarrow \emptyset$
  $ownInfo[own_0] \leftarrow \{"addr" : aggAddr, "ops" : ops, "aggKey" : aggPubKey\}$
  $opInfo \leftarrow \emptyset$
  $opInfo["addr"] \leftarrow aggAddr$
  $opInfo["keys"] \leftarrow opKeys$
  $opInfo["aggKey"] \leftarrow aggPubKey$
  $opInfo["owners"] \leftarrow [own_0]$
  return $\{"addr" : aggAddr, "ownInfo" : ownInfo, "opInfo" : opInfo\}$
end function
function CompAggKey($pubKeys$) ▷ Compute aggregate public key using MuSig2
end function
function PubKeyToAddr($pubKey$) ▷ Convert public key to Bitcoin address
end function
function GenSecRand ▷ Generate cryptographically secure random number
end function
function GenPubKey($secKey$) ▷ Generate public key from secret key
end function

Algorithm 10 Create Lock Request Transaction

Require: Owner address own_0, assets A, operator aggregate address $opAddr$, protocol fees $fees$, second chain address sc_addr
Ensure: Lock request transaction structure

function CREATELOCKREQ($own_0, A, opAddr, fees, sc\_addr$)
  $lockReq \leftarrow \{\}$
  $lockReq[\text{"}from\text{"}] \leftarrow own_0$, $lockReq[\text{"}to\text{"}] \leftarrow opAddr$, $lockReq[\text{"}locked\_assets\text{"}] \leftarrow A$
  $lockReq[\text{"}fees\text{"}] \leftarrow \{\text{"}btc\_fee\text{"} : fees[\text{"}btc\_fee\text{"}], \text{"}sc\_fee\text{"} : fees[\text{"}sc\_fee\text{"}]\}$
  $lockReq[\text{"}sc\_dest\text{"}] \leftarrow sc\_addr$
  $lockReq[\text{"}cond\text{"}] \leftarrow \{$ "op_redeem" : {"req" : "all_op_sigs","$timelock$" : $NULL$}, "own_redeem" : {"req" : "own_sig","$timelock$" : "rel_blocks_timeout"}}
  return $lockReq$
end function

Algorithm 11 Operator Setup Protocol

Require: Set of operators ops, powerset P, lock request req
Ensure: Setup components for BitVMX

function PREPOPSETUP(ops, P, req)
  $setup \leftarrow \{\text{"}tx\text{"} : \{\}, \text{"}secrets\text{"} : \{\}, \text{"}keys\text{"} : \{\text{"}ind\text{"} : \{\}, \text{"}agg\text{"} : NULL\}\}$
  for each $subset \in P$ **do**
    $subId \leftarrow$ _join(sort([$op.id$ for $op \in subset$]))
    $setup[\text{"}tx\text{"}][subId][\text{"}ops\text{"}] \leftarrow subset$
    for each $op \in ops$ **do**
      $setup[\text{"}tx\text{"}][subId][\text{"}txs\text{"}][op.id] \leftarrow$ "presigned_tx_for_" + subId
    end for
  end for
  $setup[\text{"}secrets\text{"}] \leftarrow \{op.id$: "secret_for_" + op.id for $op \in ops\}$
  return $setup$
end function

Algorithm 12 Transfer Protocol

Require: Owners own_0, own_1, operators ops, powerset P
Ensure: Transfer protocol execution

function CreateBurnTx($own_1, wrap\_asset, contract$)
  $burn \leftarrow \emptyset$
  $burn[\text{"}asset\text{"}] \leftarrow wrap\_asset$
  $burn[\text{"}action\text{"}] \leftarrow$ "burn"
  $burn[\text{"}redeem\_key\text{"}] \leftarrow own_1.ecies\_pub$
  $burn[\text{"}time\text{"}] \leftarrow$ CurrTime
  return $burn$
end function
function PrepEncShares($ops, own_1\_pub, utxo$)
  $enc\_data \leftarrow \emptyset$
  for each $op \in ops$ **do**
    $enc\_share \leftarrow$ ECIESEnc($own_1\_pub, op.key\_share, op.ecies\_priv$)
    $enc\_data[op.id] \leftarrow \{\text{"}enc\_share\text{"} : enc\_share, \text{"}pub\_key\text{"} : op.ecies\_pub\}$
  end for
  return enc_data
end function
function VerifySelectGroup(own_1, enc_shares, P)
  $valid\_ops \leftarrow \emptyset$
  for each $(op\_id, share\_data) \in enc\_shares$ **do**
    $dec\_share \leftarrow$ ECIESDec($own_1.ecies\_priv, share\_data[\text{"}enc\_share\text{"}]$)
    if VerifyShare(dec_share, share_data["pub_key"]) **then**
      $valid\_ops \leftarrow valid\_ops \cup \{op\_id\}$
    end if
  end for
  for each $group \in P$ **do**
    if group $\subseteq$ valid_ops and group $\neq \emptyset$ **then return** GroupToId($group$)
    end if
  end for
  return $NULL$
end function
function CreateSelMsg($group\_id$)
  return $\{\text{"}group\_id\text{"} : group\_id, \text{"}proof\text{"} :$ GenProof($group\_id$)$\}$
end function

Algorithm 13 Interactive Burn Protocol

Require: Constants: MAX_CERTS, MIN_SGRS, MAX_SGRS

function ValidCertChain($snap, burn\_block$)
  $chain\_val \leftarrow$ {"$snap$" : {"$height$" : $snap.height$,"agg_sig" : $ops.agg\_sig$,"$init\_state$" : $snap.state$}, "$cert\_chain$" : {"max_len" : MAX_CERTS,"$certs$" : []}, "$target$" : {"$height$" : $burn\_block.height$,"$cert$" : $burn\_block.cert$}}
  return $chain\_val$
end function
function CreateProverInput($cert\_chain$)
  $prover \leftarrow$ {"$cert\_hashes$" : [$sha256(cert)$ for $cert \in cert\_chain$], "unk_cert" : {"$sgrs$" : {"$count$" : $|cert\_chain.sgrs|$,"pub_keys" : $cert\_chain.sgr\_keys$,"agg_sig" : $cert\_chain.bls\_sig$}, "$merkle$" : {"$root$" : $cert\_chain.merkle\_root$,"$path$" : $cert\_chain.incl\_proof$}}}
  return $prover$
end function
function CreateVerifierChallenge($cert\_chain$)
  $verifier \leftarrow$ {"$chal\_opts$" : { "sgr_incl" : {"$type$" : "SGR_VERIFY","sgr_idx" : $NULL$}, "sig_valid" : {"$type$" : "BLS_SIG","$chal\_data$" : $NULL$}, "blk_incl" : {"$type$" : "BLK_CERT","blk_data" : $NULL$}}, "unk_cert" : {"seq_num" : $NULL$,"$snap\_ref$" : $NULL$}}
  return $verifier$
end function
function ExecProofVerification($prover\_in, verifier\_chal$)
  $proof \leftarrow$ {"$comps$" : {"$cert\_chain$" : $prover\_in.cert\_hashes$, "$chal\_resp$" : {"$type$" : $verifier\_chal.type$,"$proof\_data$" : $NULL$}}, "$constr$" : {"$chain\_len$" : MAX_CERTS,"sgr_count" : {"min" : MIN_SGRS,"max" : MAX_SGRS}}}
  return $proof$
end function

Ariel Futoransky. Fairgate Labs
Email address: futo@fairgate.io

Fadi Barbàra. Università di Roma La Sapienza (Italy) and Fairgate Labs

Ramsés Fernández. Fairgate Labs
Email address: ramses.fernandez@fairgate.io

Gabriel Larotonda. Universidad de Buenos Aires (Argentina) and CONICET
Email address: glaroton@dm.uba.ar

Sergio Demian Lerner. Fairgate Labs and Rootstock Labs
Email address: sergio@fairgate.io